INGEGNERIA DEL SOFTWARE / GUIDE PER L’UTENTE

ArquitecturaRepo

Manuale completo per l’utente

VERSIONE 1.4 / ITALIANO

Otto capitoli: introduzione, installazione, adozione, operatività, modifiche, recupero, accettazione e un esempio guidato.

MATÍAS ACUÑA · NOVEMBRE 2026

Documentazione verificata sulle fonti dello starter ArquitecturaRepo.

Indice

00 — Prima di iniziare **5**
Orientamento . . . 5
1. Che cosa viene installato davvero . . . 5
2. Perché serve un sistema oltre alla conversazione . . . 6
3. Responsabilità del CLIENT e dei quattro ruoli interni . . . 7
4. Evidenze, stato corrente e memoria . . . 8
5. Distinguere intenzione, modello e implementazione . . . 8
6. Un primo utilizzo proporzionato . . . 9
7. Come leggere gli otto capitoli . . . 9
8. Decisioni riservate all'utente . . . 10
9. Un esempio dall'idea all'accettazione . . . 10
10. Come misurare il valore del framework . . . 11
Fonti e limiti . . . 11

01 — Installazione e configurazione **12**
Contenuti . . . 12
1. Preparare ambiente e account . . . 12
2. Separare starter, fork e applicazione . . . 13
3. Controllare le directory prima di scrivere . . . 14
4. Scegliere il profilo corretto . . . 14
5. Creare il livello di controllo . . . 15
6. Verificare senza falsificare lo stato . . . 16
7. Avviare la scoperta del prodotto . . . 16
8. Gestire applicazioni già esistenti . . . 17
9. Connessioni e protezione dei dati . . . 17
10. Risolvere gli errori più frequenti . . . 18
Fonti e limiti . . . 18

02 — Il Planner e l'adozione del progetto **20**
Argomenti . . . 20
1. Una conversazione non è lo stato del progetto . . . 20
2. Preparare una verifica in sola lettura . . . 21
3. Distinguere domande di prodotto e scelte tecniche . . . 21
4. Confermare requisiti osservabili . . . 22
5. Stabilire baseline tecnica e piano delle consegne . . . 23
6. Effettuare un'adozione fondata su prove . . . 24
7. Verificare il passaggio a una nuova sessione . . . 24
8. Autorizzare la prima campagna ingegneristica . . . 25
9. Un colloquio concreto con il CLIENT . . . 25
10. Criteri per dichiarare pronta la baseline . . . 26
Fonti e limiti . . . 26

03 — Lavoro quotidiano e continuità dell'esecuzione **27**
Argomenti . . . 27
1. Cominciare una giornata senza ricominciare il progetto . . . 27

2. Collegare richiesta e capacità concreta . . . 28
3. Il lavoro operativo di I3 . . . 28
4. Limiti delle autorizzazioni esterne . . . 29
5. Leggere correttamente un rapporto tecnico . . . 29
6. Git e identità del lavoro . . . 30
7. Cinque domande utili ogni giorno . . . 30
8. Esempio di una giornata completa . . . 31
9. Come formulare una consegna al controller . . . 31
10. Interpretare gli stati operativi . . . 32
11. Revisione, correzioni e decisioni . . . 32
12. Lasciare un contesto recuperabile . . . 32
Fonti e limiti . . . 33

04 — Richiedere modifiche al prodotto 34
Contenuti . . . 34
1. Descrivere il risultato prima del codice . . . 34
2. Un brief breve ma sufficiente . . . 35
3. Decisioni dell’utente e del team tecnico . . . 35
4. Cambiare idea mantenendo la storia . . . 36
5. Formulare aspettative di qualità . . . 36
6. Stabilire priorità realistiche . . . 37
7. Quando una domanda al CLIENT è indispensabile . . . 37
8. Cinque esempi concreti . . . 38
9. Nuovo requisito o bug interno? . . . 38
10. Un risultato funzionante ma inutile . . . 39
11. Una conversazione di priorità . . . 39
Fonti e limiti . . . 40

05 — Errori, incidenti e recupero sicuro 41
Contenuti . . . 41
1. Conservare le prove prima di intervenire . . . 41
2. Errori di ambiente, login e permessi . . . 42
3. Problemi nella struttura di controllo . . . 42
4. Git non annulla ogni effetto . . . 43
5. Fallimenti ripetuti e cause sistemiche . . . 43
6. Sicurezza, privacy e diritti di terzi . . . 44
7. Dimostrare che il recupero è avvenuto . . . 44
8. Tre casi di recupero completi . . . 44
9. Operazioni sensibili e decisioni corrette . . . 45
10. Limiti di una validazione riuscita . . . 46
11. Feedback locale e configurazione della privacy . . . 46
Fonti e limiti . . . 46

06 — Accettazione e chiusura delle capacità 48
Argomenti . . . 48
1. Una capacità è un risultato di prodotto . . . 48
2. Cinque componenti della chiusura . . . 49
3. Identificare esattamente il candidato . . . 49
4. Lacune di prova senza nuove campagne artificiali . . . 50
5. Accettare, continuare o cambiare obiettivo . . . 50

6. Revisioni periodiche e regola di arresto . . . 51
7. Un rapporto di chiusura verificabile . . . 51
8. Un rifiuto che non obbliga a ricominciare . . . 52
9. Software utilizzabile e test dello starter . . . 52
10. Sapere quando concludere la sessione . . . 52
Fonti e limiti . . . 53

07 — Esempio completo: gestione di un’associazione 54
Percorso . . . 54
1. Il prodotto e i suoi utenti . . . 54
2. Le domande che cambiano la soluzione . . . 55
3. Un’installazione senza confusione tra repository . . . 55
4. Prima capacità: anagrafica soci . . . 56
5. Dal Planner alla campagna eseguibile . . . 57
6. Verifiche prima dell’accettazione . . . 58
7. Seconda capacità: registrare le quote . . . 58
8. Valutare l’intero percorso . . . 59
9. Primo ciclo con dati fittizi . . . 59
10. Gestire un difetto e la sua responsabilità . . . 59
11. Il secondo ciclo dipende dal primo . . . 60
12. Esercizio finale di ricostruzione . . . 60
Fonti e limiti . . . 61

00 — Prima di iniziare

ArquitecturaRepo · Manuale completo per l’utente · Edizione v1.4 · Italiano

> Questo capitolo spiega che cosa offre il framework, quali responsabilità separa e quali risultati non può garantire da solo. Si riferisce alla release pubblicata v1.4.0; per le istruzioni effettive di installazione consultare la guida pubblica corrente e le note di versione.

Orientamento

1. Che cosa viene installato davvero
2. Perché serve un sistema oltre alla conversazione
3. Responsabilità del CLIENT e dei quattro ruoli interni
4. Evidenze, stato corrente e memoria
5. Distinguere intenzione, modello e implementazione
6. Un primo utilizzo proporzionato
7. Come leggere gli otto capitoli
8. Decisioni riservate all’utente
9. Un esempio dall’idea all’accettazione
10. Come misurare il valore del framework

1. Che cosa viene installato davvero

ArquitecturaRepo è una struttura portabile per organizzare lo sviluppo del software intorno a requisiti, responsabilità, evidenze e continuità. Contiene protocolli pubblici, modelli riutilizzabili, profili di installazione, contratti verificabili e strumenti Python per convalidare e ricostruire alcuni stati. È disponibile anche un controller locale opzionale per determinati stati delle campagne di sviluppo. Non è un servizio che, appena installato, trasforma automaticamente qualunque descrizione in un’applicazione pronta.

Il punto centrale è distinguere il controllo del processo dal prodotto finale. Il framework può aiutare a registrare quali decisioni restano valide, quali modifiche sono autorizzate, quale versione del codice è stata provata e che cosa deve fare la prossima sessione. Non sostituisce la comprensione del problema, l’architettura, la programmazione effettiva, la protezione dei dati o la valutazione della qualità da parte di un essere umano.

La release pubblica v1.4.0 è identificata da un tag e da artefatti distribuiti con checksum e manifest. La presenza di una pagina più recente sul ramo main non significa che quella pagina facesse parte dello ZIP pubblicato in precedenza. Quando si installa uno starter verificabile, bisogna controllare l’identità della distribuzione effettivamente utilizzata, non soltanto la stringa della versione.

Anche i test della release hanno un ambito preciso. Possono dimostrare che

una struttura portabile si installa e si verifica nelle condizioni previste. Non dimostrano che una diversa applicazione, realizzata successivamente con il framework, sia sicura, integrata, usabile o pronta per la produzione. Gli esempi sintetici del repository sono costruiti per esercitare i meccanismi: non sono clienti reali né approvazioni operative.

Il sistema non richiede un particolare provider di intelligenza artificiale. È possibile svolgere ruoli di pianificazione e sviluppo con persone, strumenti locali o agenti esplicitamente autorizzati. L'installazione non configura in modo implicito credenziali, non abilita spese esterne e non concede il permesso di pubblicare software o dati.

2. Perché serve un sistema oltre alla conversazione

Lavorare con un assistente di programmazione in una singola chat può essere molto produttivo. Si chiede una modifica, si riceve del codice, si prova e si corregge. Questo metodo è spesso sufficiente per un esperimento piccolo e temporaneo. I problemi diventano più evidenti quando il progetto dura settimane, attraversa molte conversazioni o coinvolge più persone.

Un nuovo agente può non conoscere tutte le decisioni precedenti. Una vecchia risposta può descrivere un ramo che non è più attuale. Una funzione dichiarata completata può essere stata modificata dopo il test. Il CLIENT finisce allora per ripetere la storia del prodotto, correggere interpretazioni incoerenti e coordinare manualmente i passaggi tecnici.

ArquitecturaRepo tenta di ridurre questa dipendenza dalla memoria conversazionale attraverso superfici canoniche nel repository. Le decisioni importanti, le condizioni di autorizzazione, i riferimenti al codice e le prove devono essere ricostruibili. Una nuova sessione dovrebbe capire l'obiettivo del prodotto, le capacità già ottenute, i rischi aperti e l'azione successiva consentita senza dover leggere ogni messaggio della chat originale.

Non è corretto affermare che questo approccio sia sempre superiore a una chat diretta. Introduce un costo di organizzazione e manutenzione. Per uno script da usare una sola volta, il costo può essere maggiore del beneficio. Per un'applicazione che evolve, conserva dati o deve sopravvivere a passaggi di consegne, la tracciabilità può evitare lavoro ripetuto e decisioni sbagliate. Il confronto onesto misura risultati reali, non quantità di protocolli.

Per valutare la differenza, si può prendere la stessa piccola funzionalità e osservare non soltanto quanto rapidamente viene scritto il primo codice, ma anche come si gestisce una modifica dei requisiti, come un nuovo esecutore riprende il lavoro e quanta evidenza resta dopo l'accettazione. I due metodi possono produrre codice; l'interesse del framework è soprattutto mantenere l'evoluzione comprensibile.

3. Responsabilità del CLIENT e dei quattro ruoli interni

Il CLIENT rappresenta il proprietario dell'intenzione di prodotto. Decide quale problema risolvere, chi utilizzerà il sistema, quali comportamenti sono importanti, quale qualità percepita è accettabile e quali limiti economici o di pubblicazione esistono. Può respingere un'interfaccia tecnicamente funzionante se risulta confusa. Non dovrebbe però dover risolvere conflitti Git o progettare i test per permettere al team di continuare.

Il framework distingue quattro classi interne di responsabilità:

Ruolo	Responsabilità principale	Limite
I1 Master Owner, quando necessario	Coerenza tra prodotti, risorse condivise e problemi sistemici	Non diventa il Planner di ogni piccola attività
I2 Planner	Interpretazione del prodotto, strategia, architettura e criteri di accettazione	Non gestisce i singoli tentativi di debugging
I3 Execution Controller	Implementazione, test, integrazione, recupero e revisioni nella campagna autorizzata	Non inventa nuove decisioni di prodotto
I4 Specialisti e revisori	Lavoro tecnico circoscritto e verifiche indipendenti	Non sostituiscono autorizzazioni strategiche o del CLIENT

Quattro classi di ruolo non significano quattro modelli di intelligenza artificiale o quattro processi sempre attivi. Un progetto semplice può avere una sola persona che svolge diverse funzioni, mantenendo però chiare le responsabilità e cercando una revisione realmente indipendente dove il rischio lo richiede.

La distinzione decisiva è tra progettare una capacità e realizzarla. I2 comprende il problema, stabilisce confini e criteri, poi autorizza una campagna. Da quel momento I3 gestisce il lavoro concreto: correzioni, test, refactoring interno, integrazione e recupero. Un bug ordinario non deve generare una nuova campagna né obbligare il CLIENT a scegliere l'algoritmo di correzione.

Un cambiamento sostanziale dei requisiti è diverso. Se il CLIENT trasforma una piccola applicazione locale in un servizio pubblico su Internet, bisogna rivalutare autenticazione, sicurezza, dati personali, distribuzione e costi. Non sarebbe responsabile trattarlo come un semplice dettaglio della campagna esistente.

4. Evidenze, stato corrente e memoria

Per stabilire che cosa esiste tecnicamente, si parte da prove fisiche: file, Git, configurazione, database, artefatti e stato di runtime. Poi si controllano le superfici canoniche correnti, i requisiti confermati e le autorizzazioni attive. Un riepilogo di chat può orientare la ricerca, ma non prevale su evidenze tecniche più recenti.

L'intenzione del CLIENT segue una regola complementare. Una decisione esplicita successiva può sostituire una preferenza precedente. La nuova scelta deve però essere riportata nella descrizione corrente del prodotto, con una supersessione comprensibile. Né un documento vecchio né un assistente che indovina possono sostituire una decisione riservata all'utente.

Quando una prova fondamentale manca, il valore corretto è sconosciuto, non PASS. Se due fonti attive si contraddicono, la modifica materiale che dipende dal conflitto si ferma fino alla riconciliazione. La diagnosi in sola lettura può proseguire. Il sistema deve saper dire con precisione che cosa non sa.

Il livello di controllo generato si chiama normalmente .agentic-sdlc/. Il suo stato iniziale HOLD / UNTRUSTED_CONTEXT è intenzionale. Significa che i dati di esempio non sono stati scambiati per realtà del progetto. Solo una scoperta verificata e un'adozione coerente permettono di promuovere i fatti necessari a uno stato operativo.

L'attualità di un dato non deriva dal suo nome. Una cartella chiamata CURRENT può contenere un riferimento obsoleto. Un commit appena creato può non coincidere con il database realmente in uso. Currentness è il legame dimostrabile fra autorità, codice, configurazione, dati e prove nel momento in cui si pretende di lavorare o accettare un risultato.

5. Distinguere intenzione, modello e implementazione

Per leggere il progetto senza confondere piani diversi sono utili tre livelli. L0 raccoglie ciò che il CLIENT ha realmente chiesto, chiarito o autorizzato. L1 contiene l'interpretazione confermata: requisiti attuali, modello del prodotto e Product State. L2 raccoglie il codice, i database, le campagne, gli esiti dei test e gli altri artefatti tecnici.

Una funzione può essere ben descritta in L1 e mancare ancora in L2. Oppure può essere implementata in L2 ma non corrispondere al significato richiesto in L0. In entrambi i casi non basta un messaggio di successo. Il team deve poter risalire dalla capacità osservabile alla decisione che l'ha giustificata e al candidato che la realizza.

Questa separazione permette anche di individuare una promessa prematura. «L'applicazione gestisce i pagamenti» può significare soltanto che esiste una schermata dimostrativa, oppure che vengono registrati contributi locali, oppure che vengono realmente eseguite transazioni esterne. Sono capacità molto diverse e richiedono autorizzazioni e prove differenti.

Il modello corrente dovrebbe dire quale di queste affermazioni è stata confermata e quale no. Una buona architettura rende le differenze visibili

invece di nasconderle sotto etichette generiche come pronto o completato.

6. Un primo utilizzo proporzionato

Per iniziare, scegliete il profilo più piccolo adeguato ai rischi. SMALL è spesso sufficiente per un esperimento senza complessità persistente; STATEFUL è più adatto quando l'applicazione conserva dati importanti; altri profili trattano artefatti pesanti, più workstream, più prodotti o conseguenze particolarmente rilevanti.

Il primo passo non è avviare decine di agenti. È scrivere una breve descrizione del prodotto e verificare il repository reale. Chi sono gli utenti? Quale azione devono poter compiere? Quali dati sono coinvolti? Quale ambiente verrà usato? Che cosa è escluso dalla prima versione? Che cosa resta da decidere?

Il Planner ricostruisce poi il codice esistente, propone la base tecnica e organizza una prima capacità utile. Prima di autorizzare il lavoro, si verifica che una nuova sessione possa comprendere i riferimenti correnti senza l'intera conversazione originale. In mancanza di questa continuità, conviene correggere le informazioni alla fonte invece di produrre altro codice sulla base di ipotesi.

La guida di installazione è il riferimento pratico per i comandi; il capitolo 01 li spiega in dettaglio. Non è necessario memorizzare subito l'intero manuale per provare un profilo semplice.

7. Come leggere gli otto capitoli

Capitolo	Quando consultarlo	Risultato atteso
00 · Prima di iniziare	Prima dell'adozione	Comprendere scopo, responsabilità e limiti
01 · Installazione	Quando si prepara un progetto	Verificare la distribuzione e creare il controllo iniziale
02 · Planner e adozione	Prima della prima capacità	Ricostruire requisiti e base tecnica attuale
03 · Lavoro quotidiano	Durante lo sviluppo	Continuare nella campagna autorizzata
04 · Richiedere modifiche	Quando cambiano i bisogni	Esplicitare effetto, priorità e confini
05 · Errori e recupero	Dopo un incidente	Preservare prove e recuperare con sicurezza
06 · Accettazione	Quando qualcuno dice «finito»	Valutare candidato, gate e qualità umana

Capitolo	Quando consultarlo	Risultato atteso
07 · Esempio di associazione	Per esercitare l’intero ciclo	Seguire un prodotto fittizio dall’idea alla verifica

Le otto mini guide e le FAQ restano utili come introduzione breve. Non diventano automaticamente la traduzione completa di questo libro. Il manuale spagnolo v1.3 viene conservato separatamente come fonte storica e non viene riscritto per simulare il comportamento della v1.4.

8. Decisioni riservate all’utente

Un buon brief iniziale può essere molto semplice: «Voglio un’applicazione locale per registrare i soci e le quote della nostra associazione. La userà un amministrativo. Non voglio pagamenti online nella prima versione. La cronologia delle quote deve restare disponibile e l’interfaccia deve essere leggibile».

Queste informazioni orientano il prodotto senza prescrivere l’architettura. Il CLIENT può aggiungere limiti di tempo, accessibilità, budget e uso di servizi esterni. Può anche lasciare aperte questioni sulle quali non ha ancora deciso. La squadra tecnica deve chiarire solo quelle che cambiano materialmente il risultato.

Non serve che il CLIENT scelga il database, il framework web, il numero di branch o la struttura delle classi. Engineering propone e giustifica tali scelte in relazione al contesto. Se una tecnologia impone conseguenze importanti per costi, privacy o uso, queste vengono spiegate prima di richiedere la decisione riservata.

9. Un esempio dall’idea all’accettazione

L’associazione vuole eliminare una gestione fragile dei soci. Il CLIENT conferma che i pagamenti storici non devono sparire quando una persona viene disattivata. Il Planner osserva prima il repository, verifica se esistono dati e applicazioni precedenti, poi definisce una prima capacità che consenta di creare e ritrovare un socio.

I3 implementa il percorso completo, scrive test e gestisce le correzioni. Un test trova duplicati; il controller indaga la causa, corregge e verifica di nuovo nello stesso charter. Il Planner non deve ridisegnare la campagna a ogni bug, e il CLIENT non deve spiegare come creare un vincolo di unicità.

Quando il candidato è pronto, vengono controllati separatamente il comportamento del codice, l’integrazione con i dati, la situazione reale di esecuzione e la comprensibilità dell’interfaccia. Il CLIENT valuta ciò che gli compete: se la gestione dei soci è realmente comoda e coerente con il bisogno espresso.

Un resoconto positivo da solo non chiude la capacità. L'accettazione deve riferirsi a una versione identificabile e a prove adeguate. Il capitolo 07 sviluppa lo scenario con due capacità collegate: soci e quote.

10. Come misurare il valore del framework

Dopo alcuni cicli, chiedete una dimostrazione breve: quale nuova capacità funziona, quale rischio è stato ridotto, quale prova sostiene l'affermazione e quale azione successiva è consentita? Se la risposta consiste soltanto in più documenti, agenti o commit, il valore per il prodotto non è ancora stato dimostrato.

Il framework può migliorare la continuità e la distribuzione delle responsabilità. Non elimina giudizio, test reali, sicurezza, privacy o valutazione umana. Un progetto maturo sa fermarsi quando le condizioni concordate sono soddisfatte e sa dichiarare HOLD quando un'incertezza importante impedisce una modifica sicura.

L'obiettivo finale resta un software concretamente utile, costruito e verificato con responsabilità chiare. Le regole hanno valore se rendono questo risultato più affidabile e meno dipendente dalla memoria di una singola conversazione.

Segue: capitolo 01, Installazione.

Fonti e limiti

- Release v1.4.0
- Avvio pubblico v1.4
- Note di rilascio e upgrade
- Panoramica dell'architettura
- Modello delle autorità
- Licenza MIT

Il capitolo descrive il framework e un caso fittizio; non certifica un'applicazione di terzi né autorizza servizi esterni.

01 — Installazione e configurazione

ArquitecturaRepo · Manuale completo per l’utente · Edizione v1.4 · Italiano

> Questo capitolo distingue l’installazione dello starter dalla sua adozione in un progetto reale. I comandi sono allineati alla guida pubblica v1.4. Il successo del programma di installazione non dimostra che l’applicazione sia stata capita, implementata o accettata.

Contenuti

1. Preparare ambiente e account
2. Separare starter, fork e applicazione
3. Controllare le directory prima di scrivere
4. Scegliere il profilo corretto
5. Creare il livello di controllo
6. Verificare senza falsificare lo stato
7. Avviare la scoperta del prodotto
8. Gestire applicazioni già esistenti
9. Connessioni e protezione dei dati
10. Risolvere gli errori più frequenti

1. Preparare ambiente e account

Gli esempi usano Windows, PowerShell, Git e Python. Non è obbligatorio utilizzare esattamente questi strumenti per beneficiare dell’architettura. Serve però un modo affidabile per identificare la versione del codice, accedere ai file del progetto e avviare gli strumenti di verifica compatibili con la distribuzione scelta.

La versione Python 3.12 è stata esercitata nei test portabili della release v1.4. Per controllare l’interprete attivo in PowerShell si possono eseguire questi comandi:

```
& "python" --version
& "py" --version
```

Se non funzionano, controllate l’installazione dell’interprete e il PATH del terminale. Non scaricate eseguibili da pagine sconosciute per risolvere frettolosamente un errore. Un comando fallito va diagnosticato nel contesto reale: sistema operativo, directory, permessi e dipendenze.

Un servizio di coding remoto può usare risorse esterne, ma non per questo l’applicazione locale richiede una GPU particolare. Allo stesso modo, una potente macchina locale non concede automaticamente accesso a un modello

esterno. Il framework non installa modelli, non seleziona provider e non autorizza consumi a pagamento per il semplice fatto di essere presente.

Prima di procedere, scegliete il repository dell'applicazione. È facile aprire la cartella dello starter in un editor e chiedere a un agente di realizzare lì il prodotto. In questo modo la documentazione pubblica, gli esempi e il codice dell'applicazione rischiano di mescolarsi. Il percorso di destinazione deve essere verificato e distinto.

Proteggete gli account collegati. Usate autenticazione adeguata e il minimo accesso necessario. Un collegamento GitHub non dimostra che l'assistente stia guardando il repository corretto; una sessione autenticata non equivale a permesso di effettuare pubblicazioni o modifiche distruttive.

2. Separare starter, fork e applicazione

Ci sono tre oggetti diversi che possono sembrare lo stesso progetto:

Oggetto	Scopo	Che cosa non rappresenta
Repository ArquitecturaRepo	Protocolli, strumenti, profili, modelli, test e documentazione pubblica	L'applicazione da sviluppare
Fork dello starter, eventuale	Personalizzazioni mantenute dall'utilizzatore	Un Product State reale creato automaticamente
Repository dell'applicazione	Codice, dati, configurazione e livello .agentic-sdlc/ installato	Una copia indiscriminata degli esempi sintetici

Una struttura locale sensata può essere:

```
C:\Progetti\
    ArquitecturaRepo\
    gestione-soci\
```

Il nome della cartella è soltanto illustrativo. La separazione delle responsabilità è invece reale: lo starter fornisce meccanismi generici; gestione-soci contiene il prodotto specifico. Il progetto può rimanere privato anche quando il framework usato è pubblico.

Se si intende modificare e mantenere una variante del framework, un fork può essere utile. In caso contrario basta un checkout attendibile o l'archivio della release. Non è necessario creare un repository nuovo ogni volta che una conversazione riparte. Anzi, cloni duplicati e worktree dimenticati possono rendere ambigua la versione corrente e consumare spazio.

Per una distribuzione riproducibile, consultate la release v1.4.0 e controllate checksum e manifest del pacchetto. Il ramo main può ricevere documentazione

successiva alla pubblicazione. Non bisogna descrivere tali aggiunte come contenuto dello ZIP già pubblicato.

3. Controllare le directory prima di scrivere

Prima dell'installazione aprite il terminale nel checkout del framework e verificate dove siete:

```
Get-Location
& "python" --version
& "git" status --short
```

Get-Location mostra la directory corrente; il controllo dell'interprete conferma l'ambiente; git status rende visibili le modifiche locali. Se il comando Git fallisce, non date per scontato che il progetto sia danneggiato: potreste semplicemente trovarvi fuori dal repository.

Se compaiono modifiche esistenti, non rimuovetele per rendere il tutorial più semplice. Possono appartenere a un altro sviluppatore oppure rappresentare lavoro ancora non integrato. La prima responsabilità è conservarle e comprenderne la relazione con la versione attuale.

Verificate il percorso dell'applicazione separatamente. Editor, terminale, tool di coding e client Git possono puntare a cartelle diverse. Prima di concedere scrittura a un agente, chiarite in quale directory opererà e quali risorse potrà modificare.

Se l'applicazione contiene già .agentic-sdlc/, un controller, Product State, campagne o dati di esercizio, non tentate una seconda installazione sopra la prima. La procedura di attachment per progetti già governati parte da una verifica in sola lettura. Non promette una migrazione generale e automatica da qualsiasi versione precedente.

4. Scegliere il profilo corretto

Il profilo va scelto in relazione ai rischi e alle caratteristiche del prodotto, non al numero di agenti che si desidera lanciare. I profili pubblici sono definiti in profiles.json.

Profilo	Quando considerarlo
SMALL	Prodotto circoscritto e rischio operativo limitato
STATEFUL	Dati persistenti, utenti, backup, migrazioni e ripresa
ARTIFACT_HEAVY	Molti output, file grandi, provenienza, diritti e gestione dello spazio
MULTI_WORKSTREAM	Flussi di lavoro diversi che devono integrarsi nello stesso prodotto

Profilo	Quando considerarlo
MULTI_PRODUCT	Prodotti distinti che condividono risorse o governance
HIGH_CONSEQUENCE	Conseguenze elevate su sicurezza, privacy, finanza o azioni irreversibili

Un semplice prototipo può iniziare con SMALL. Un gestionale che conserva dati di soci e quote deve considerare STATEFUL. Questo non impone automaticamente un Master Owner separato o una rete di molti modelli. I ruoli sono responsabilità, non un requisito numerico di processi.

Documentate brevemente il motivo della scelta. Un profilo non certifica che i rischi siano già stati mitigati e non concede nuove autorizzazioni. Se il prodotto cambia, i controlli possono essere adattati sulla base di evidenze aggiornate, senza trasformare ogni piccola modifica in una ricostruzione totale.

5. Creare il livello di controllo

Dal checkout verificato dello starter v1.4, in PowerShell, un nuovo progetto non governato può essere preparato così:

```
& "python" "tools\scaffold_project.py" --target "..\mio-progetto" --
profile "SMALL" --product-id "MIO_PROGETTO" --objective "Obiettivo iniziale da co
-language "it" --no-feedback-workflow
```

Per il caso immaginario dell'associazione, con dati persistenti, una configurazione motivata potrebbe essere:

```
& "python" "tools\scaffold_project.py" --target "..\gestione-soci" --
profile "STATEFUL" --product-id "GESTIONE_SOCI" --objective "Anagrafica soci e qu
-language "it" --no-feedback-workflow
```

Sono due esempi alternativi. Non eseguiteli entrambi sulla stessa destinazione. target indica il percorso esatto; profile seleziona i controlli; product-id è un identificatore del progetto; objective è una descrizione iniziale ancora da confermare; language it seleziona la documentazione localizzata.

Il programma crea il livello .agentic-sdlc/ nella destinazione, mantenendo i file applicativi e i workflow preesistenti. Deve rifiutare la sovrascrittura di un controllo canonico già installato. Non esiste una regola generale secondo cui ripetere l'installazione risolve un problema di validazione.

Lo stato iniziale HOLD / UNTRUSTED_CONTEXT è corretto. Il framework non ha ancora verificato quale codice, database, requisito o decisione sia attuale nel progetto. Modificare manualmente questo stato in READY per ottenere un esito verde falsificherebbe la continuità.

L'opzione –with-runtime aggiunge meccanismi locali opzionali. Non installa un provider esterno, non autentica una sessione Codex Desktop e non attiva

automaticamente campagne reali. I permessi e i gate di esecuzione sono autonomi rispetto alla creazione delle cartelle.

6. Verificare senza falsificare lo stato

Dopo l'installazione, aprite il terminale nella cartella dell'applicazione, non in quella del framework. Il percorso iniziale di verifica comprende:

```
& "python" -m pip install -r ".agentic-sdlc\requirements-validation.txt"
&   "python"   ".agentic-sdlc\tools\project_status.py"   --control-
root ".agentic-sdlc"
&   "python"   ".agentic-sdlc\tools\discover_project.py"   --project-
root "."
& "python" ".agentic-sdlc\tools\reconstruct_context.py" --root ".agentic-
sdlc"
```

La prima riga installa le dipendenze dichiarate per la validazione. Le successive verificano lo stato riconosciuto, raccolgono una descrizione del progetto senza modificare il codice e tentano di ricostruire il contesto. Il risultato HOLD nella ricostruzione è normale finché non esiste una base adottata e confermata.

Controllate tre aspetti fisici: il livello .agentic-sdlc/ si trova davvero nella destinazione prevista; i file e i workflow originali non sono stati sostituiti; gli strumenti restituiscono risultati interpretabili e riproducibili. Se manca jsonschema, installate i requisiti nell'interprete effettivamente utilizzato.

La validazione dei contratti è distinta dalla prova della vostra applicazione. Potete avere file JSON impeccabili e un gestionale che non riesce a salvare un socio. Allo stesso modo, un'applicazione può avviarsi mentre lo stato canonico indica erroneamente una campagna vecchia. I diversi livelli di verifica non si sostituiscono.

Conservate il risultato del primo controllo e il riferimento alla versione del framework. Se un nuovo esecutore torna sul progetto, deve poter capire se una determinata difficoltà riguarda l'ambiente, l'adozione o il codice del prodotto.

7. Avviare la scoperta del prodotto

La tappa successiva è la discovery, non l'implementazione immediata di funzionalità. Il prompt di installazione e adattamento, il prompt di discovery e il contratto di adozione chiariscono quali informazioni debbano essere raccolte.

Il Planner controlla il repository reale, le tecnologie, la situazione dei dati, gli utenti, le funzionalità presenti e i rischi operativi. Il CLIENT conferma i requisiti di prodotto. Engineering verifica e decide la base tecnica, pianifica una prima consegna e prepara l'adozione delle informazioni coerenti nel Product State e negli altri archivi previsti.

Non copiate la configurazione sintetica di un esempio come se descrivesse il vostro sistema. Una SHA inventata, un Goal fittizio o un'approvazione mai data non diventano veri perché rispettano uno schema. Se il dato manca, si deve conservare l'incertezza e cercare la prova corretta.

L'adozione avviene dopo un preflight. La ricostruzione in un contesto nuovo deve poter identificare obiettivo, requisiti, versione del codice, stato dei dati, responsabilità e prossima azione autorizzata. Soltanto dopo è ragionevole avviare la prima campagna materiale.

8. Gestire applicazioni già esistenti

Molti utenti non partono da zero. Un'applicazione potrebbe contenere codice reale, migrazioni e dati preziosi prima dell'installazione del framework. In questo caso l'operazione deve rispettare la realtà esistente: nessuna pulizia indiscriminata, nessuna cancellazione della cronologia, nessuna sostituzione cieca del controllo.

Distinguete un progetto completamente nuovo, un progetto con codice ma senza governance canonica e un progetto già governato. Nel primo caso si segue l'installazione ordinaria. Nel secondo si effettua prima una verifica del percorso e degli artefatti. Nel terzo si usa il preflight in sola lettura per capire compatibilità, conflitti e possibili rischi.

Le note di rilascio non devono essere interpretate come prova di una migrazione automatica da qualsiasi versione. Il percorso esatto dipende dai contratti e dallo stato realmente osservati. È meglio mantenere temporaneamente un HOLD ben motivato che perdere le informazioni necessarie alla ripresa.

9. Connessioni e protezione dei dati

Un account GitHub o un servizio di coding può essere utile per leggere e modificare il repository, ma i suoi permessi vanno controllati. Non concedete accessi generali semplicemente perché potrebbero servire in futuro. Un comando locale reversibile non equivale all'autorizzazione a pubblicare, cancellare dati reali, inviare messaggi o acquistare servizi.

Non copiate token, password o chiavi nei prompt, nelle Issue pubbliche o nel codice versionato. Se una diagnosi richiede log, eliminate i valori sensibili senza perdere l'informazione utile. Lo starter pubblico non richiede il trasferimento del database privato dell'applicazione ai manutentori.

Nella v1.4 è previsto un feedback architetturale locale con metadati limitati. L'opzione –no-feedback-workflow disattiva la pianificazione periodica del workflow GitHub Actions, ma non il generatore locale. Per disabilitare ogni report impostate enabled=false nella configurazione .agentic-sdlc/feedback/ARCHITECTURE_FEEDBACK_CONFIG.json. Consultate la descrizione dei controlli privacy.

Generazione locale, pianificazione e trasmissione esterna sono operazioni

diverse. Non si deve presumere che l'esistenza di un file di report implichi l'invio a terzi, né che l'assenza di un workflow impedisca l'esecuzione manuale del generatore.

10. Risolvere gli errori più frequenti

Sintomo	Significato possibile	Intervento sicuro
Python non riconosciuto	Interprete assente o PATH errato	Controllare terminale e installazione
Cartella .agentic-sdlc/ non presente	Target sbagliato o comando fallito	Leggere output e percorso esatto
HOLD / UNTRUSTED_CONTEXT	Requisiti e stato reale non ancora adottati	Procedere con discovery
jsonschema mancante	Dipendenze non installate nell'ambiente attivo	Usare requirements-validation.txt
Controller esistente preservato	Protezione contro sovrascrittura	Seguire attachment in sola lettura
Ricostruzione incoerente	Riferimenti di codice, stato o autorità incompatibili	Preservare evidenze e riconciliare
Tool collegato al repo sbagliato	Workspace o permessi errati	Sospendere le scritture e verificare
Test verdi, app inutilizzabile	Prove del controllo, non del prodotto	Avviare l'applicazione e provare il flusso

Una diagnosi utile classifica il problema prima di intervenire: ambiente, percorso, permessi, dipendenze, compatibilità, schema, stato corrente o reale funzionalità. Ripetere a caso l'installazione e cancellare directory importanti può peggiorare la situazione.

Considerate completato questo capitolo quando l'installazione giusta è verificabile nel percorso previsto, il codice applicativo preesistente è intatto e lo stato iniziale espone onestamente ciò che ancora non si sa. Il capitolo successivo collega il controllo alle esigenze e al codice reali.

Segue: capitolo 02, Planner e adozione.

Fonti e limiti

- Release v1.4.0
- Guida pubblica v1.4
- Note di rilascio
- Fork e bootstrap
- Contratto di trasferimento e adozione
- Attachment per progetti governati

- Feedback e privacy
- Codice dello scaffold

Le istruzioni riguardano lo starter. Non dimostrano l’esecuzione o l’accettazione di un’applicazione esterna.

02 — Il Planner e l’adozione del progetto

ArquitecturaRepo · Manuale completo per l’utente · Edizione v1.4 · Italiano

> L’installazione crea gli strumenti di controllo, non una conoscenza valida del prodotto. Questo capitolo illustra come ricostruire la realtà, ottenere la conferma del CLIENT e adottare uno stato coerente. Le fonti sono le regole di product discovery e il contratto di adozione.

Argomenti

1. Una conversazione non è lo stato del progetto
2. Preparare una verifica in sola lettura
3. Distinguere domande di prodotto e scelte tecniche
4. Confermare requisiti osservabili
5. Stabilire baseline tecnica e piano delle consegne
6. Effettuare un’adozione fondata su prove
7. Verificare il passaggio a una nuova sessione
8. Autorizzare la prima campagna ingegneristica
9. Un colloquio concreto con il CLIENT
10. Criteri per dichiarare pronta la baseline

1. Una conversazione non è lo stato del progetto

Un assistente può aver discusso per ore una buona architettura, ma la discussione non dimostra che il codice esista o sia aggiornato. Un messaggio precedente può sostenere che il database ha già certe tabelle mentre il repository attuale contiene una migrazione diversa. Una schermata mostrata al cliente può appartenere a una versione poi modificata. Il Planner deve distinguere desideri, decisioni confermate, proposte e stato fisico.

La prima domanda non è «quali funzionalità programmiamo oggi?», bensì «che cosa sappiamo realmente del prodotto e da quali fonti?». Vanno ricostruiti l’obiettivo, gli utenti, il codice corrente, i dati, le integrazioni, i rischi e le autorizzazioni. Se esiste una governance precedente, il Planner deve leggerla e rispettarne i confini prima di aggiungere nuovi meccanismi.

Il CLIENT non deve diventare il custode della coerenza tecnica. Può confermare le esigenze e correggere l’interpretazione di un flusso di lavoro, ma non dovrebbe dover ricostruire SHA, migrazioni o responsabilità di un controller. Sono attività tecniche da svolgere consultando le evidenze disponibili.

Quando una fonte manca, l’assenza va dichiarata. È meglio un HOLD preciso che una falsa ricostruzione con identificatori plausibili. Riempire un modello JSON con un hash inventato o copiare il Product State di un esempio sintetico

non rappresenta un'adozione: significa creare una descrizione tecnicamente coerente nella forma ma falsa nel contenuto.

2. Preparare una verifica in sola lettura

Un buon primo passaggio è sistematico ma proporzionato. Si identifica il repository effettivo, il ramo di lavoro, le modifiche locali, i file di configurazione, i test e i comandi di avvio. Si controlla quali database siano realmente collegati, quali migrazioni siano applicate e se esistano artefatti rilevanti al di fuori del codice.

La verifica iniziale non dovrebbe cancellare file né generare decine di copie del repository. Se esiste un checkout attendibile, lo si riutilizza. Per confrontare il lavoro con la sorgente remota si aggiornano i riferimenti necessari e si analizza il delta, senza interpretare automaticamente il movimento Git come identità o autorizzazione di un agente.

Una traccia utile comprende:

1. Prodotto, utenti e principali percorsi d'uso.
2. Ultima intenzione di prodotto confermata.
3. Identità del codice e stato delle modifiche non integrate.
4. Stack, database, ambiente, rete e distribuzione.
5. Permessi, segreti e servizi esterni coinvolti.
6. Controlli già installati, Product State e campagne.
7. Test fisici disponibili e loro rapporto con il codice corrente.
8. Incertezze che cambiano la prossima azione possibile.

La sessione di product discovery e il protocollo di baseline aiutano a dare ordine alla verifica. Non devono però produrre un grande archivio di moduli ripetitivi. Ogni informazione importante deve avere un proprietario canonico e una superficie corrente chiaramente identificabile.

3. Distinguere domande di prodotto e scelte tecniche

Il CLIENT definisce ciò che l'utente deve ottenere. Il team tecnico decide come realizzarlo. La divisione sembra semplice, ma nel lavoro quotidiano è facile confonderla. Chiedere al CLIENT quale database utilizzare può essere improprio quando la vera domanda riguarda il numero di persone che useranno il programma contemporaneamente.

Domanda al CLIENT	Scelta che spetta all'ingegneria
Quanti operatori modificano i dati allo stesso tempo?	Concorrenza, transazioni, conflitti
Che cosa significa disattivare un socio?	Schema, riferimenti e migrazioni

Domanda al CLIENT	Scelta che spetta all’ingegneria
Serve accesso da Internet oppure solo locale?	Topologia, porte, autenticazione e distribuzione
Chi può consultare lo storico delle quote?	Ruoli e controllo degli accessi
Quanto tempo si può perdere dopo un guasto?	Strategia di backup, recupero e ripresa
La prima versione prevede pagamenti reali?	Integrazioni esterne dopo autorizzazione distinta

Domande migliori producono decisioni utilizzabili. Per esempio «dobbiamo conservare i pagamenti dopo la disattivazione?» chiarisce un comportamento concreto. La risposta influenza l’architettura, ma non obbliga il cliente a definire chiavi esterne o indici.

Il Planner segnala anche requisiti in conflitto. Se il cliente vuole eliminazione definitiva di ogni dato e contemporaneamente conservazione completa di tutte le operazioni economiche, le due aspettative devono essere chiarite alla luce delle regole applicabili. Non si dovrebbe implementare una soluzione arbitraria e soltanto dopo comunicarne le conseguenze.

Le decisioni più sensibili riguardano sicurezza, dati personali, pubblicazione e operazioni esterne. Engineering può spiegare rischi e opzioni; la parte riservata al CLIENT deve essere confermata in modo esplicito e tracciabile.

4. Confermare requisiti osservabili

Dopo il confronto, il Planner produce una rappresentazione compatta dei requisiti attuali. Non basta una frase generica come «gestire i soci». Occorre sapere quali operazioni sono necessarie, quale comportamento è corretto in condizioni normali e che cosa deve succedere nei principali casi limite.

Per l’associazione del nostro esempio, una prima proposta potrebbe essere:

Dimensione	Proposta per la prima versione	Stato
Utente	Un amministrativo su un computer dell’associazione	Da confermare
Soci	Creare, trovare, modificare e disattivare	Da confermare
Quote	Registrare importo, data e legame con il socio	Da confermare
Storico	Mantenere le quote dopo la disattivazione	Da confermare
Rete	Uso locale, senza accesso pubblico	Da confermare

Dimensione	Proposta per la prima versione	Stato
Interfaccia	Moduli chiari e messaggi comprensibili	Da confermare
Recupero	Definire frequenza dei backup e perdita tollerata	Decisione aperta
Esclusioni	Niente pagamenti online nella prima versione	Da confermare

Questa tabella non inventa una conferma. Presenta proposte che il CLIENT può accettare o modificare. Una volta confermate, le decisioni diventano l'autorità di prodotto corrente. Una modifica futura deve essere registrata come supersessione, mantenendo chiara la provenienza della scelta.

Un requisito ben formulato contiene anche una condizione di verifica. «Ritrovare un socio dopo aver riavviato il programma» è osservabile. «Usare un'architettura robusta» è un obiettivo tecnico importante, ma deve essere tradotto in controlli specifici da chi ne è responsabile.

La documentazione dovrebbe distinguere gli aspetti confermati, quelli dedotti da evidenze tecniche e quelli ancora aperti. Non serve trasformare ogni piccola scelta di interfaccia in una nuova campagna strategica, ma i comportamenti che possono causare perdita di dati o alterare il prodotto non devono restare impliciti.

5. Stabilire baseline tecnica e piano delle consegne

Una volta chiarita la direzione del prodotto, il Planner definisce la baseline tecnica. Deve contenere informazioni verificabili sullo stack, sulle versioni rilevanti, sulla persistenza, sui confini di sicurezza e rete, sulle modalità di esecuzione e sugli obblighi di recupero. Le preferenze personali non sono sufficienti se contraddicono il codice o l'ambiente effettivo.

Per una piccola applicazione locale una base dati incorporata potrebbe essere adeguata. Se in seguito il prodotto richiede modifiche simultanee da molti utenti, serve una nuova valutazione. La scelta corretta considera concorrenza, capacità, integrità, manutenzione e costo, non soltanto la familiarità dell'agente con una tecnologia.

Il piano delle consegne ordina capacità effettivamente utili. Per esempio: prima anagrafica soci, poi quote, poi elenco degli arretrati. Ogni fase ha dipendenze e criteri riconoscibili. Non occorre anticipare tutto il possibile sviluppo futuro né inventare una data di rilascio che il CLIENT non ha confermato.

Priorità e rischio si influenzano. Una migrazione potenzialmente distruttiva richiede prove e strategia di recupero più approfondite. Una modifica visiva può essere meno rischiosa sul piano dei dati, ma richiedere una verifica

umana dell'esperienza. La pianificazione deve identificare quali controlli sono veramente necessari per la capacità scelta.

Gli schemi e strumenti di validazione controllano la forma della baseline. Non certificano automaticamente la verità dei valori. Un documento perfettamente valido può contenere l'hash sbagliato o descrivere un ambiente che nessuno ha mai avviato. La verifica delle fonti fisiche resta indispensabile.

6. Effettuare un'adozione fondata su prove

Adottare il progetto significa promuovere informazioni verificate in uno stato di controllo utilizzabile. È un passaggio distinto dallo scaffold. Prima si osserva, poi si conferma, infine si applica una specifica di adozione coerente.

Il preflight confronta riferimenti, file, identità, obblighi e stato esistente. Se una superficie canonica è già governata da un altro sistema, l'operazione non deve sovrascriverla automaticamente. Se manca una prova di currentness, il sistema deve segnalarla e fermare soltanto ciò che dipende materialmente da quel dato.

Gli strumenti discover_project.py, apply_adoption.py e reconstruct_context.py coprono fasi diverse. Discovery raccoglie elementi osservabili, adoption promuove la configurazione verificata e reconstruction prova che lo stato corrente sia recuperabile.

Le chiamate precise dipendono dalla versione installata e dalle istruzioni contenute nel suo repository. Non si dovrebbero copiare comandi di una release precedente senza confrontarli. La presenza di un pulsante o una voce di menu in un editor non sostituisce la verifica dell'effetto reale.

Se l'adozione fallisce, conservate il rapporto e diagnosticate il motivo. Un conflitto di proprietà, una referenza a una migrazione assente e un requisito non approvato richiedono interventi diversi. Correggete la causa sulla superficie proprietaria e ripetete la verifica pertinente. Non create un secondo Product State concorrente.

7. Verificare il passaggio a una nuova sessione

Il test più importante di questo capitolo è eseguire una ricostruzione in un contesto fresco. Un nuovo esecutore, privo della memoria della chat originale, deve identificare:

- scopo e utenti del prodotto;
- requisiti correnti confermati e decisioni aperte;
- fonte precisa del codice e stato dei dati;
- baseline tecnica e vincoli di distribuzione;
- responsabile delle decisioni strategiche e operative;
- eventuale campagna autorizzata e suo gate corrente;
- evidenza più forte disponibile per le capacità esistenti;
- prossima azione consentita e operazioni ancora vietate.

Se la risposta richiede al CLIENT di raccontare di nuovo tutto, il trasferimento non è ancora valido. La soluzione è migliorare la superficie canonica mancante, non produrre una nuova sintesi priva di riferimenti. A volte basta un collegamento corretto a un file già esistente; altre volte serve riconciliare dati realmente contraddittori.

Anche l'informazione negativa è preziosa. «La funzione è stata testata solo con dati fittizi», «nessun provider esterno è autorizzato» oppure «la migrazione di produzione resta sconosciuta» sono fatti che impediscono a un futuro agente di compiere un'azione pericolosa.

La ricostruzione deve essere ripetibile. Se due sessioni, leggendo le stesse fonti correnti, raggiungono versioni incompatibili della realtà, c'è un difetto di ownership o di riferimento da risolvere.

8. Autorizzare la prima campagna ingegneristica

Con requisiti e baseline adottati, il Planner può definire il primo charter. Prima considera il prodotto end-to-end, i punti di dipendenza, i dati, le interfacce, i rischi, i limiti operativi e i criteri di accettazione. Una prima analisi strategica viene seguita da una revisione avversariale distinta, adatta all'importanza della decisione.

Il charter descrive una capacità: che cosa si vuole ottenere, perché, quali confini sono autorizzati, chi possiede la realizzazione, quali interfacce devono restare coerenti, quali gate servono e quali fatti invaliderebbero la decisione. Non è un elenco rigido di micro-task.

Una prima capacità sensata è «registrare un socio con dati obbligatori, assegnargli un'identità stabile e ritrovarlo dopo il riavvio». Si può consegnare e verificare come un percorso completo, senza dover costruire in anticipo ogni funzione contabile futura.

Una volta autorizzato il charter, I3 coordina implementazione, test, debugging e integrazione. I2 torna per la verifica strategica terminale o quando emerge una vera modifica dei confini. Un errore di validazione all'interno del charter non richiede una nuova analisi strategica totale.

9. Un colloquio concreto con il CLIENT

Il CLIENT dice: «Vorrei sostituire i fogli di calcolo con un piccolo programma per i soci e le quote. Lo userà una persona e per ora deve restare locale. Non voglio abbonamenti esterni».

Il Planner risponde prima sul problema: «Confermo che la prima necessità è registrare e ritrovare un socio in modo affidabile. Che cosa deve succedere allo storico se il socio viene disattivato? Quali dati sono obbligatori? Quale recupero vi serve se il computer si guasta?».

Il CLIENT chiarisce le regole. Il Planner verifica il repository e i dati disponibili, propone la baseline tecnica e mostra un piano con capacità successive. Il CLIENT conferma il lato funzionale, mentre Engineering

sceglie implementazione e test.

Prima di avviare la campagna, una nuova sessione ricostruisce il contesto direttamente dal repository. Se manca un collegamento essenziale, lo si corregge. Soltanto dopo la prima capacità viene affidata al controller con limiti e verifiche chiari.

10. Criteri per dichiarare pronta la baseline

La baseline è sufficientemente pronta quando un lettore esterno sa quale prodotto costruire, quali requisiti siano stati confermati, quale codice e ambiente siano reali, chi possa prendere la prossima decisione e quale lavoro sia effettivamente permesso.

Non è necessario anticipare ogni futuro dettaglio. È necessario non nascondere le incertezze materiali. Un elenco infinito di documenti non sostituisce la prova di continuità. La qualità della baseline si misura dalla capacità di guidare la prima consegna senza confusione e di resistere al passaggio a un altro contesto.

Se il processo diventa troppo pesante per una piccola applicazione, riducete la complessità organizzativa mantenendo le regole essenziali: intenzione confermata, fonte corrente, responsabilità, prova e prossima azione. La proporzione è parte dell'architettura, non un compromesso vergognoso.

Segue: capitolo 03, Lavoro quotidiano.

Fonti e limiti

- Discovery, requisiti e baseline
- Trasferimento e adozione
- Planner Kernel
- Sessione di discovery
- Strumento di discovery
- Strumento di adozione
- Ricostruzione del contesto

Le proposte relative all'associazione sono illustrative. Non costituiscono requisiti confermati per un altro progetto.

03 — Lavoro quotidiano e continuità dell’esecuzione

ArquitecturaRepo · Manuale completo per l’utente · Edizione v1.4 · Italiano

> Dopo l’adozione e l’autorizzazione di una campagna, il lavoro deve restare coerente anche quando cambiano persone, sessioni o strumenti. Questo capitolo segue l’Execution Team OS, il contratto del charter e le regole di currentness.

Argomenti

1. Cominciare una giornata senza ricominciare il progetto
2. Collegare richiesta e capacità concreta
3. Il lavoro operativo di I3
4. Limiti delle autorizzazioni esterne
5. Leggere correttamente un rapporto tecnico
6. Git e identità del lavoro
7. Cinque domande utili ogni giorno
8. Esempio di una giornata completa
9. Come formulare una consegna al controller
10. Interpretare gli stati operativi
11. Revisione, correzioni e decisioni
12. Lasciare un contesto recuperabile

1. Cominciare una giornata senza ricominciare il progetto

Il primo passo non dovrebbe essere rileggere migliaia di messaggi. Chi riprende un progetto apre il repository effettivo, controlla la versione corrente e ricostruisce l’autorità di lavoro: requisiti attivi, Product State, campagna, checkpoint, prove disponibili e azione successiva permessa.

Una conversazione precedente può essere utile, ma non è automaticamente attuale. Se un vecchio rapporto dichiara completato un modulo, il nuovo esecutore deve verificare se quel risultato si riferisce al codice che esiste ora. Se la banca dati è cambiata, la sola identità Git non prova che la migrazione sia ancora valida.

La ricerca deve essere proporzionata. Con un checkout affidabile si possono leggere le differenze recenti e i riferimenti canonici, senza riclonare tutto. Uno stato locale modificato va preservato finché non si conoscono proprietario e significato delle modifiche. Anche i worktree temporanei e gli artefatti pesanti devono avere uno scopo preciso.

Un buon riepilogo operativo contiene pochi fatti controllabili: «La campagna

attiva riguarda la ricerca soci; il candidato esatto è identificato; il gate aperto è l'integrazione con il database; l'azione permessa è eseguire test e correzioni nello stesso charter; non è autorizzata la pubblicazione». Un nuovo agente dovrebbe poter verificare ciascuna parte.

2. Collegare richiesta e capacità concreta

Il CLIENT può chiedere «voglio trovare facilmente i soci». Per definire una capacità utile, I2 chiarisce che cosa significa facilmente: ricerca per nome o numero, gestione degli omonimi, persone disattivate, errori e dimensione dei dati. Non deve necessariamente decidere da solo ogni dettaglio; può fare domande di prodotto mirate.

Il risultato non coincide con un singolo componente tecnico. Un endpoint di ricerca può funzionare isolatamente ma restare inutilizzabile se la schermata non mostra correttamente i risultati. Una capacità end-to-end include l'ingresso dell'utente, le verifiche di autorizzazione, la lettura dei dati e una risposta comprensibile.

Il charter descrive questo risultato e i confini del lavoro. Non impone in anticipo tutte le classi, le query o i commit. La decomposizione tecnica spetta a I3 dopo l'autorizzazione. In questo modo la campagna può assorbire normali difficoltà di implementazione senza ritornare continuamente alla pianificazione.

Non è necessario costruire una ricerca universale per soddisfare un bisogno limitato. Se la prima versione richiede unicamente il numero di socio, tale limite può essere esplicito. La qualità dipende dalla corrispondenza tra comportamento concordato e risultato reale, non dal numero massimo di funzionalità possibili.

3. Il lavoro operativo di I3

I3 gestisce implementazione, debugging, integrazione, verifiche, eventuali migrazioni interne, ripresa dopo un'interruzione e cicli di revisione già compresi nel charter. Può organizzare specialisti I4 o eseguire direttamente compiti tecnici in un progetto piccolo. La responsabilità resta continua fino a un candidato realmente verificabile.

Supponiamo che la ricerca dei soci non funzioni con lettere accentate. Questo è normalmente un difetto tecnico rispetto a un requisito già confermato. Il controller esamina codifica, normalizzazione, dati e query; corregge la causa e ripete i test. Non serve creare un nuovo Goal né chiedere al CLIENT quale algoritmo usare.

Se la stessa classe di errore ricompare molte volte, il problema non si risolve aggiungendo infinitamente casi speciali. I3 deve esaminare il sottosistema in profondità, rivedere l'ipotesi tecnica e ottenere una verifica indipendente. Un nuovo passaggio strategico è necessario solo se emergono nuove condizioni materiali che invalidano la decisione iniziale.

Un cambio di prodotto è diverso: se il CLIENT vuole rendere pubblica su Internet una ricerca finora locale, cambiano rischi, autorizzazioni e forse architettura. Questa richiesta torna al proprietario della decisione e al Planner. I3 non deve espandere il proprio charter senza una base valida.

4. Limiti delle autorizzazioni esterne

La presenza di credenziali o connessioni non equivale a permesso generale. Consultare un file locale, eseguire test sintetici e generare una bozza sono azioni diverse da modificare un database di produzione, inviare email reali, utilizzare un servizio a pagamento o pubblicare contenuti protetti.

Ogni azione sensibile dovrebbe avere un obiettivo preciso, un limite e un'autorità riconoscibile. «Finisci tutto» non autorizza spesa illimitata né trattamento indiscriminato di dati personali. Un agente può proporre un'operazione esterna, ma non deve dedurne automaticamente il consenso del CLIENT.

Per esempio, «puoi provare la migrazione sulla copia sintetica e verificare il backup; non toccare la produzione» è un confine utile. Se non è chiaro se una transazione reale sia già stata eseguita, il controller non la ripete alla cieca. Prima ricostruisce il fatto e cerca una ricevuta idempotente quando il sistema ne dispone.

Per le risorse di terzi, il progetto deve rispettare licenze e condizioni concrete. Un controllo ragionevole dei diritti non significa pretendere certificazioni inutili per ogni file, ma nemmeno considerare liberamente pubblicabile tutto ciò che è tecnicamente scaricabile.

5. Leggere correttamente un rapporto tecnico

Un rapporto «tutti i test verdi» è un indizio, non una prova completa. Occorre sapere quali test, quale codice, quale database, quali condizioni e quali requisiti. È importante che la dichiarazione di successo resti legata al candidato esatto che è stato provato.

Un rapporto utile separa affermazioni e prove:

- Quale capacità viene dichiarata disponibile?
- Quale versione concreta è stata eseguita?
- Quali controlli hanno prodotto un risultato osservabile?
- Quali casi o ambienti sono rimasti fuori?
- Quali errori sono stati corretti e quali rischi restano?
- Chi può prendere la decisione di accettazione?

Una registrazione della schermata può mostrare il flusso visivo, ma non necessariamente la correttezza delle transazioni. Un test unitario può dimostrare il comportamento di una funzione, ma non la leggibilità dell'interfaccia. Un hash di un pacchetto identifica il contenuto, non ne certifica la sicurezza o l'utilità.

Il tempo dei test conta. Se il codice viene modificato dopo una revisione materiale, i vecchi risultati potrebbero non essere più applicabili. Il controller deve rivalutare i gate coinvolti e fornire una nuova prova. Questo non è inutile perfezionismo: impedisce di accettare un candidato diverso da quello esaminato.

6. Git e identità del lavoro

Git è fondamentale per riconoscere il codice, ma non registra automaticamente tutte le autorizzazioni. Un commit può essere creato da uno strumento per conto di una persona; il suo autore tecnico non prova necessariamente l'identità dell'agente o la validità della campagna. Le informazioni di responsabilità devono essere mantenute separatamente quando necessario.

Non esiste neppure una regola secondo cui due lavori dello stesso repository debbano sempre fermarsi a vicenda. La concorrenza va decisa sulle risorse realmente toccate: codice condiviso, database, migrazioni, configurazioni o artefatti. Due workstream diversi possono interferire sulla stessa tabella; due modifiche allo stesso prodotto possono essere indipendenti.

Quando si creano worktree temporanei per una concorrenza legittima, bisogna rimuoverli una volta completato il loro scopo. Non accumulate cloni, cache, render o screenshot per sicurezza indefinita. Conservate gli artefatti che hanno reale valore probatorio, con percorso, versione e provenienza riconoscibili.

Una riparazione non dovrebbe partire da comandi distruttivi. Se il working tree contiene modifiche non note, indagatele. Il reset forzato non è un metodo universale per eliminare errori e può cancellare lavoro o prove non ancora integrate.

7. Cinque domande utili ogni giorno

Alla fine di ogni sessione il team può rispondere a cinque domande. Primo: quale capacità utilizzabile è avanzata? Secondo: quale candidato concreto e quali prove lo dimostrano? Terzo: quale rischio o gate resta aperto? Quarto: chi possiede la prossima decisione? Quinto: la prossima sessione potrà continuare senza consultare tutto il vecchio dialogo?

Le risposte possono essere brevi. Non servono lunghe trascrizioni quotidiane. L'importante è distinguere avanzamento reale da attività. Dieci nuovi commit non sono un risultato se la funzione principale continua a fallire. Un bug di integrità corretto e verificato può invece rappresentare un miglioramento sostanziale anche senza nuove schermate.

Quando il lavoro è bloccato da un'incertezza, il rapporto deve indicare l'esatto fatto mancante. «Non possiamo modificare la banca dati finché non sappiamo quale migrazione è stata applicata» è una conclusione utile. Nel frattempo, un'analisi di codice indipendente può ancora essere permessa.

Queste cinque domande non sostituiscono i test formali. Servono a mantenere

il lavoro orientato al prodotto e a rendere visibile una decisione futura senza ricreare l'intera storia.

8. Esempio di una giornata completa

La mattina il controller apre il gestionale dell'associazione. Il charter autorizza la ricerca dei soci per nome e numero. Il repository e la base dati sintetica vengono identificati; nessuna modifica alla produzione è permessa. I test esistenti verificano che la registrazione dei soci funzioni ancora.

Durante lo sviluppo emerge un errore: i soci disattivati non appaiono più nei risultati, anche quando l'utente applica il filtro storico. La regola confermata dice invece che la cronologia deve restare consultabile. I3 corregge la query, aggiunge casi di regressione e prova il percorso dalla schermata al database.

Un revisore indipendente esamina i casi difficili: nessun risultato, molti omonimi, campi vuoti, caratteri speciali e accesso non autorizzato. L'esperienza umana viene provata separatamente. Se l'utente non capisce come attivare il filtro, la funzione non soddisfa ancora pienamente la qualità attesa.

La sera il team registra il candidato effettivo, i risultati tecnici, gli eventuali limiti e la prossima azione. Il Planner rientra per una vera decisione strategica, non per rilasciare nuove istruzioni su ogni piccolo fix. Così la continuità non dipende dalla memoria della giornata.

9. Come formulare una consegna al controller

Una consegna tecnica efficace può essere breve ma completa:

> Obiettivo: rendere utilizzabile end-to-end la ricerca soci concordata. Fonti: stato corrente verificato e charter in vigore. Limiti: nessun dato di produzione, nessun provider esterno, nessuna modifica del significato di disattivazione. Verifiche: unità, integrazione, database, comportamento reale e usabilità come gate distinti. Conclusione: candidato esatto, prove, falsificatori e problemi rimasti.

L'ordine di lavoro non ripete l'intera conversazione precedente se le informazioni sono già nel repository. Indica invece le responsabilità e i vincoli che permettono a I3 di lavorare in modo autonomo. In questo contesto, autonomo significa gestire i dettagli tecnici autorizzati, non inventare preferenze dell'utente.

La consegna deve evitare microtask eccessivamente prescrittivi. Se il controller scopre che un refactoring interno è necessario per la correttezza, può eseguirlo entro il confine del charter. Se rileva una nuova esigenza di pubblicazione o una dipendenza esterna non prevista, deve fermare l'azione pertinente e richiedere la decisione corretta.

10. Interpretare gli stati operativi

Il controller locale opzionale campaignctl.py può gestire alcuni passaggi e checkpoint. Le sue transizioni rappresentano lo stato della superficie che controlla; non dimostrano da sole che un agente remoto abbia eseguito codice né che la produzione sia stata aggiornata correttamente.

In caso di interruzione si controllano identità, ultimo checkpoint valido e attività esterne già compiute. Una ripresa non deve ripetere automaticamente un effetto irreversibile. È necessario distinguere un comando non avviato da uno completato ma non ancora registrato nella conversazione.

Se uno stato risulta incoerente, non si dovrebbe correggerlo assegnando casualmente un nuovo identificatore. La diagnosi individua la causa sulla fonte proprietaria. La decisione di continuare o mantenere HOLD dipende dai fatti verificati e dall'effetto della modifica programmata.

11. Revisione, correzioni e decisioni

Il rapporto terminale deve descrivere l'intera capacità e non soltanto l'ultimo file modificato. Un revisore controlla il candidato esatto. Se trova difetti già compresi nel charter, questi tornano a I3 come lavoro da correggere nella stessa campagna.

Il Planner valuta se la capacità realizzata corrisponde alla strategia e ai criteri concordati. Il CLIENT conserva la valutazione del risultato visibile e le autorizzazioni riservate. Un test tecnico non diventa automaticamente accettazione umana, così come una demo gradevole non sostituisce la verifica della banca dati.

Se due o più ritorni riguardano lo stesso sottosistema, I3 deve fare hardening complessivo e revisione indipendente prima di proporre un altro candidato. Questa regola evita la produzione infinita di rapporti prematuri e promuove la soluzione delle cause.

La qualità dell'esecuzione si riconosce dal risultato verificabile, non dal numero di agenti coinvolti o di ore di attività. Anche il rapporto più dettagliato resta insufficiente se il candidato non può essere riprodotto.

12. Lasciare un contesto recuperabile

Prima di chiudere, aggiornate le superfici canoniche pertinenti: codice o candidato, stato della campagna, gate aperti, prove e prossima azione legale. Una fonte storica può rimanere accessibile, ma non dovrebbe confondersi con l'autorità corrente.

Evitate di copiare le stesse politiche in molti documenti. Una regola stabile ha un proprietario; uno stato che cambia ha una superficie corrente; un rapporto di test conserva il proprio contesto. Collegamenti chiari e versioni riconoscibili sono più affidabili di un enorme riassunto senza provenienza.

La prova finale è chiedere a un’altra sessione di aprire il repository e continuare in sicurezza. Se deve rivolgersi al CLIENT per risolvere ogni dettaglio tecnico, il processo non ha ancora mantenuto la promessa di continuità.

Segue: capitolo 04, Richiedere modifiche.

Fonti e limiti

- Execution Team OS
- Charter di campagna
- Currentness e ricostruzione
- Efficienza e igiene del repository
- Integrità del review
- Controller locale opzionale

Il percorso descritto non conferma esecuzione nativa di servizi esterni o accettazione di un prodotto concreto.

04 — Richiedere modifiche al prodotto

> Il CLIENT deve poter formulare desideri e correzioni senza assumersi il lavoro di progettazione tecnica. Questo capitolo segue il confine delle decisioni del CLIENT e il processo di definizione dei requisiti.

Contenuti

1. Descrivere il risultato prima del codice
2. Un brief breve ma sufficiente
3. Decisioni dell’utente e del team tecnico
4. Cambiare idea mantenendo la storia
5. Formulare aspettative di qualità
6. Stabilire priorità realistiche
7. Quando una domanda al CLIENT è indispensabile
8. Cinque esempi concreti
9. Nuovo requisito o bug interno?
10. Un risultato funzionante ma inutile
11. Una conversazione di priorità

1. Descrivere il risultato prima del codice

Quando si lavora con un agente di programmazione è naturale chiedere direttamente una soluzione: «Aggiungi una nuova tabella», «usa Redis» oppure «riscrivi il servizio». Il problema è che queste richieste tecniche possono nascondere una necessità più semplice. Se il vero obiettivo è trovare un socio rapidamente, può bastare migliorare l’indice esistente invece di introdurre un servizio aggiuntivo.

Il punto di partenza migliore è il comportamento osservabile. Chi usa la funzione? Che cosa prova a fare? Quale risultato deve ottenere? In quali condizioni oggi fallisce? Quali parti del prodotto non devono cambiare? La risposta permette al Planner di scegliere una capacità completa e al Controller di decidere l’implementazione.

Un cambiamento ben descritto non deve essere lungo. «L’amministrativo deve poter filtrare le quote per mese, vedere il totale e correggere un errore di input senza perdere i dati già inseriti» è già molto più chiaro di «rifai la tabella delle quote».

Questo metodo non vieta al CLIENT di imporre vincoli tecnici. Se l’applicazione deve funzionare offline o integrarsi con una piattaforma imposta da un contratto, sono requisiti legittimi. Semplicemente, non bisogna trasferirgli le scelte ordinarie di classi, query e protocolli quando non sono

decisioni di prodotto.

2. Un brief breve ma sufficiente

Una buona richiesta di modifica comprende quattro parti: contesto, risultato, limiti e modo di verificare. Può aggiungere priorità e data se queste cambiano realmente il lavoro.

Esempio:

> Come responsabile amministrativo voglio consultare tutte le quote scadute di un mese. L'elenco deve indicare socio, importo e scadenza, senza mostrare a utenti non autorizzati i contatti personali. Non devono essere modificati i pagamenti già registrati. Nella prima versione non serve esportare in Excel; desidero prima verificare che i numeri siano corretti.

Il Planner può tradurre questa richiesta in requisiti, dipendenze e criteri di accettazione. I3 decide il percorso tecnico: query, servizio, interfaccia, test e gestione degli errori. La separazione permette di correggere i dettagli senza richiedere nuove istruzioni al CLIENT ogni volta.

Se non è ancora chiaro come gestire un caso particolare, indicatelo come domanda aperta. Per esempio: «Non so se le quote pagate in ritardo debbano ancora apparire nello storico degli arretrati; spiegatemi le conseguenze prima di decidere». Una domanda aperta è meglio di un comportamento inventato in autonomia.

Le richieste più rischiose devono aggiungere informazioni sui dati e sugli effetti esterni. Un nuovo invio automatico di email reali richiede una valutazione diversa da una notifica interna simulata. Un'esportazione con nomi e contatti necessita di controlli di autorizzazione e gestione sicura del file.

3. Decisioni dell'utente e del team tecnico

Il CLIENT decide che cosa conta per l'utilizzatore: regole, visibilità, priorità, qualità percepita, costi e pubblicazione. Engineering risponde delle scelte tecniche e delle verifiche: architettura, persistenza, migrazioni, logging, error handling, test e recupero.

Un cliente che chiede un backup deve descrivere quanto storico vuole proteggere e quale perdita recente sia tollerabile. Non dovrebbe dover decidere se usare snapshot, journal o una particolare chiamata di filesystem. Il team tecnico progetta una soluzione coerente con il prodotto e spiega le eventuali conseguenze su tempo e budget.

Se le scelte tecniche presentano alternative che cambiano il comportamento visibile, la decisione torna al CLIENT. Una sincronizzazione eventuale tra più dispositivi, per esempio, potrebbe mostrare dati non immediatamente aggiornati. Questo effetto è una questione di prodotto, anche se viene originato da un compromesso architetturale.

Lo stesso vale per permessi sensibili. Il fatto che una API sia accessibile o che un token esista non autorizza invii reali, pagamenti, pubblicazione di dati personali o operazioni irreversibili. Le richieste devono essere interpretate alla luce delle responsabilità e dei limiti confermati.

4. Cambiare idea mantenendo la storia

Durante lo sviluppo si scoprono bisogni che all'inizio non erano evidenti. Il CLIENT può cambiare direzione. Forse un prodotto pianificato per un solo computer deve ora funzionare in due sedi. Forse un report non risponde alla domanda del consiglio direttivo. Questo apprendimento è normale e deve poter aggiornare il prodotto.

La modifica va registrata come nuova decisione corrente. Il precedente requisito resta una fonte storica, ma non deve più guidare silenziosamente il lavoro quando è stato sostituito. Il Planner valuta le conseguenze sui dati, sul codice, sulle campagne attive e sul piano di consegna.

Una nuova decisione importante può invalidare il charter. Passare da un'applicazione locale a un servizio Internet modifica autenticazione, deployment, sicurezza e disponibilità; non dovrebbe essere trattato come un semplice refactoring interno. Serve una rivalutazione strategica appropriata.

Al contrario, una correzione del colore di una notifica già previsto come parte della qualità della stessa schermata può restare nel charter esistente. Non è necessario creare una campagna nuova per ogni dettaglio quando l'obiettivo resta invariato.

Non cancellate indiscriminatamente documenti precedenti. Una tracciabilità delle supersessioni consente di capire perché una scelta era valida ieri e non lo è oggi. Questo riduce contraddizioni apparenti e aiuta una nuova sessione a ricostruire la vera intenzione attuale.

5. Formulare aspettative di qualità

La qualità non coincide soltanto con test automatici. Un'interfaccia può salvare correttamente un socio ma richiedere troppi passaggi, usare termini incomprensibili o rendere difficili gli errori più comuni. Il CLIENT può chiedere un risultato leggibile, coerente, accessibile e facile da imparare.

Una buona richiesta può essere: «Una persona che non conosce la struttura del database deve riuscire a inserire un socio senza istruzioni speciali. I campi obbligatori devono essere evidenti, e in caso di dati duplicati il sistema deve spiegare come procedere».

Questa formulazione chiarisce un gate percettivo senza prescrivere il framework grafico. Engineering definisce e realizza l'implementazione. La persona incaricata della valutazione dell'interfaccia prova il percorso reale. Un test unitario non sostituisce la verifica che l'utente capisca una schermata.

L'aspettativa può riguardare anche affidabilità: «Nessun pagamento deve comparire due volte in seguito a un doppio click». Questo comportamento va convertito in controlli di idempotenza, persistenza e integrazione. Il CLIENT definisce la semantica, I3 la tecnica.

Non serve rendere ogni preferenza un criterio matematico. Alcuni giudizi richiedono una vera valutazione umana. È però utile spiegare concretamente il difetto osservato e conservare un candidato identificabile per verificare la correzione.

6. Stabilire priorità realistiche

Aprire dieci Goal contemporanei per dieci desideri non produce necessariamente una consegna più rapida. Le richieste possono condividere dati, configurazione e risorse, oppure dipendere l'una dall'altra. Il Planner deve mostrare la sequenza che porta più rapidamente a una capacità utilizzabile senza trascurare rischi reali.

Supponiamo che l'associazione chieda categorie di soci, promemoria email, un riepilogo delle quote e una nuova homepage. Il riepilogo potrebbe dipendere dalle categorie; l'invio email coinvolge permessi e dati; la homepage può essere quasi indipendente se non modifica gli stessi componenti.

Il CLIENT assegna priorità dal punto di vista dell'utilità: ciò che serve davvero prima della data obiettivo, ciò che è importante ma rinviabile, ciò che resta fuori dalla prima versione. Il Planner integra le dipendenze tecniche e propone un percorso realistico.

Una prima consegna non deve essere soltanto un prototipo finto che sembra completo. Può essere volutamente piccola, ma il suo comportamento essenziale deve funzionare, compresa la gestione degli errori e la persistenza quando necessaria. Una piccola capacità reale fornisce feedback migliore di una grande promessa senza prove.

Il principio di proporzione vale anche per la documentazione. Non si dovrebbe spendere più tempo a creare nuove campagne e passaggi formali che a risolvere le incertezze che contano. L'obiettivo è sbloccare valore, non massimizzare il numero di artefatti amministrativi.

7. Quando una domanda al CLIENT è indispensabile

Una domanda è indispensabile quando una scelta di prodotto non confermata cambia materialmente ciò che verrà realizzato. Eliminare definitivamente un socio o conservarlo come disattivato è un esempio. Permettere o meno l'accesso pubblico a un report con dati personali è un altro.

Non è invece necessario interrogare il CLIENT per un test di regressione fallito, una correzione di parsing o una migrazione interna già compresa nel charter. Questi casi rientrano nella responsabilità tecnica, purché non introducano nuove conseguenze importanti.

Una buona domanda presenta le alternative dal punto di vista dell'utente: «Se

nascondiamo i soci disattivati, la ricerca quotidiana è più semplice, ma per vedere le quote storiche dovrai aprire l'archivio. Se li lasciamo visibili con un filtro, avrai un elenco più lungo ma accesso immediato. Quale flusso è migliore per il tuo lavoro?».

Il CLIENT può decidere senza conoscere query SQL o classi Java. Il team tecnico si assume la responsabilità della traduzione nel sistema. Questa è una distinzione strutturale, non una questione di cortesia.

Se due requisiti confermati sono incompatibili, il Planner deve esplicitare il conflitto. Non deve scegliere arbitrariamente quello più facile da implementare. Una scelta tracciabile evita che in seguito il comportamento del prodotto venga giustificato con una presunta decisione mai presa.

8. Cinque esempi concreti

Ricerca soci: «L'operatore deve trovare una persona per nome o codice. I soci inattivi devono essere consultabili tramite un filtro e la ricerca non deve alterare i dati». Il comportamento è verificabile senza imporre un indice specifico.

Pagamento quota: «Dopo aver registrato una quota, l'operatore deve vederla nello storico. Un doppio click non deve creare due pagamenti». Questa richiesta rende esplicito un rischio di integrità da trattare tecnicamente.

Disattivazione: «Un socio disattivato non può prenotare nuove attività; i pagamenti precedenti restano consultabili». La regola collega più capacità e richiede coerenza del modello.

Report del consiglio: «Il report mensile deve mostrare importi e conteggi senza esporre inutilmente indirizzi personali. Solo gli autorizzati possono aprirlo». Il vincolo riguarda sia utilità sia privacy.

Interfaccia: «Le operazioni quotidiane devono essere riconoscibili senza termini tecnici; in caso di errore il messaggio deve spiegare che cosa correggere». Questo enunciato porta a un gate di verifica umana.

Il Planner può trasformare ciascun esempio in criteri tecnici di accettazione. Si devono osservare casi positivi, limiti, permessi, persistenza e qualità pertinente. I dettagli implementativi restano nella competenza di Engineering.

9. Nuovo requisito o bug interno?

Un bug è un comportamento che non soddisfa la decisione già confermata. Se la ricerca deve trattare correttamente i nomi con accenti e non lo fa, I3 corregge nel charter esistente. Una nuova richiesta cambia invece il risultato autorizzato. Collegare la ricerca a una banca dati esterna introduce un nuovo confine.

Un difetto ripetuto non diventa automaticamente un nuovo prodotto. Se cinque correzioni superficiali falliscono, il controller deve investigare in modo sistemico il sottosistema e ottenere una revisione indipendente. Nuovi

Planner PASS non riparano di per sé problemi di implementazione.

Al contrario, una decisione sul trattamento di pagamenti reali, su un nuovo deployment pubblico o sulla cancellazione irreversibile di dati deve essere portata all'autorità giusta. Chiamarla «piccola modifica tecnica» non riduce il rischio né crea il permesso mancante.

La classificazione serve anche al tempo del CLIENT. Una governance ben costruita non gli restituisce bug ordinari da coordinare. Gli presenta domande concrete quando una decisione personale o di business è effettivamente necessaria.

10. Un risultato funzionante ma inutile

Può accadere che il codice sia corretto e che la funzione non risolva il problema. Un report può calcolare perfettamente la somma delle quote pagate, mentre il consiglio voleva vedere quelle scadute. Un'interfaccia può salvare ogni modifica ma richiedere un percorso troppo complesso per la routine quotidiana.

Il CLIENT può rifiutare il risultato e spiegare che cosa manca: «Posso trovare le informazioni, ma ho bisogno di vedere subito le quote in ritardo, senza aprire ogni socio». Questa osservazione non è un difetto di test unitario; riguarda il significato e l'esperienza dell'applicazione.

Il Planner determina se il comportamento era già previsto dai requisiti o se si tratta di una nuova necessità emersa durante l'uso. La distinzione decide se I3 deve correggere all'interno della campagna o se serve una nuova scelta strategica.

Una valutazione umana non va falsificata per ottenere un PASS. Se l'utente non ha mai provato il flusso, la qualità percettiva resta non dimostrata. Allo stesso tempo non bisogna prolungare indefinitamente una campagna alla ricerca di difetti microscopici dopo che i criteri concordati sono davvero soddisfatti.

11. Una conversazione di priorità

Per preparare una modifica, bastano spesso cinque domande: chi vive il problema, quale risultato deve cambiare, quali dati o regole vanno conservati, quale priorità domina e quale decisione ancora aperta può modificare sostanzialmente il lavoro.

Una volta raccolte queste informazioni, il Planner verifica il codice corrente, analizza le dipendenze e formula una capacità ragionevole. I3 realizza e prova l'intervento. Il CLIENT interviene per approvare il comportamento e giudicare l'esperienza, non per prescrivere ogni tentativo di debug.

Se emergono esigenze diverse, registratele con il loro stato: confermate, proposte o escluse. Una lista di desideri non dovrebbe trasformarsi silenziosamente in dieci promesse di consegna. Ogni capacità deve avere confini comprensibili e una prova applicabile.

La buona richiesta di modifica tutela sia la libertà tecnica sia l’autorità dell’utente. Permette a chi sviluppa di scegliere la soluzione adeguata e a chi userà il prodotto di riconoscere quando l’esito desiderato è stato raggiunto.

Segue: capitolo 05, Errori e recupero.

Fonti e limiti

- Confine del CLIENT
- Product discovery e baseline
- Planner Kernel
- Piano di consegna e requisiti
- Charter di campagna
- Ciclo di vita delle capacità

Le richieste illustrate riguardano il caso fittizio dell’associazione e non costituiscono decisioni per un altro prodotto.

05 — Errori, incidenti e recupero sicuro

ArquitecturaRepo · Manuale completo per l’utente · Edizione v1.4 · Italiano

> Quando qualcosa non funziona, l’obiettivo è capire la causa e recuperare senza perdere prove o dati. Questo capitolo integra le regole di ricostruzione, il protocollo di recupero e i confini di sicurezza.

Contenuti

1. Conservare le prove prima di intervenire
2. Errori di ambiente, login e permessi
3. Problemi nella struttura di controllo
4. Git non annulla ogni effetto
5. Fallimenti ripetuti e cause sistemiche
6. Sicurezza, privacy e diritti di terzi
7. Dimostrare che il recupero è avvenuto
8. Tre casi di recupero completi
9. Operazioni sensibili e decisioni corrette
10. Limiti di una validazione riuscita
11. Feedback locale e configurazione della privacy

1. Conservare le prove prima di intervenire

Un errore è un’informazione. La risposta immediata non dovrebbe essere cancellare il repository, reinstallare il framework o forzare un reset. Prima si registra che cosa è successo, su quale versione, con quali dati e durante quale operazione. Il messaggio di errore completo, l’ambiente, il commit, il database e gli eventuali checkpoint servono a distinguere causa e sintomo.

La pausa necessaria dipende dall’effetto possibile. Se manca una dipendenza Python, si possono continuare altre attività indipendenti. Se una migrazione ha forse modificato i dati reali dei soci, ulteriori scritture sul database devono fermarsi finché non si conosce lo stato. Non è necessario bloccare automaticamente ogni parte del progetto, ma nemmeno procedere come se il rischio non esistesse.

Le operazioni di recupero devono avere una destinazione precisa. Un reset indiscriminato del working tree può cancellare modifiche di altri esecutori. Una nuova migrazione può duplicare un effetto già applicato. Cancellare log e file temporanei prima di classificarli può rimuovere gli unici indizi dell’incidente.

Una sequenza affidabile è osservare, preservare, classificare, formulare un’ipotesi, applicare una correzione autorizzata e verificare l’effetto. Se le prove non permettono di stabilire quale transazione sia stata completata, lo

stato resta sconosciuto. La certezza non può essere prodotta scrivendo una dichiarazione rassicurante.

2. Errori di ambiente, login e permessi

Molti problemi non derivano dal codice dell'applicazione. L'interprete può essere quello sbagliato, una libreria non essere installata, il percorso del repository essere diverso da quello immaginato oppure la sessione di autenticazione essere scaduta.

Cominciate dalle condizioni osservabili. Controllate il percorso corrente, la versione del linguaggio, le dipendenze dichiarate e i permessi. Se un agente non vede i file, non concedetegli subito accesso a tutte le cartelle: verificate prima quale workspace gli è stato effettivamente aperto.

Un errore di accesso non autorizza a superare una protezione. Se l'operazione richiede un'autenticazione o un permesso aggiuntivo, si coinvolge il proprietario del servizio secondo il flusso previsto. Non è corretto raccogliere credenziali fuori dal contesto autorizzato o riutilizzare segreti di un'altra applicazione.

Quando un provider esterno non risponde, distinguete rete, quota, fatturazione, credenziali, versione API e bug. Un test locale con dati sintetici può restare possibile anche se l'operazione reale è temporaneamente vietata. La diagnosi dovrebbe esplicitare questa separazione.

3. Problemi nella struttura di controllo

Il livello .agentic-sdlc/ contiene riferimenti a requisiti, Product State, ruoli e campagne. Un controllo che fallisce non significa sempre che la struttura sia rotta. Potrebbe segnalare correttamente che non si conosce la fonte corrente di una decisione o che due stati sono in contraddizione.

Gli strumenti project_status.py, validate_project.py e reconstruct_context.py aiutano a vedere queste condizioni. Un errore di sintassi JSON è diverso da una SHA valida nella forma ma riferita a codice vecchio. Nel primo caso serve una correzione di formato; nel secondo serve una ricostruzione della realtà.

Non riempite campi con identificatori inventati per ottenere un PASS. Un valore plausibile può essere più pericoloso di un campo dichiarato sconosciuto, perché porta gli esecutori successivi a compiere azioni basate su una falsa certezza. La documentazione deve distinguere ciò che è osservato da ciò che è proposto.

Se una campagna risulta conclusa ma i test non sono collegati al candidato attuale, la sua accettazione non è dimostrata per quel candidato. Il controller deve recuperare o produrre la prova adeguata, non dichiarare che la revisione precedente vale automaticamente per ogni modifica futura.

Anche il nome CURRENT non è una garanzia magica. La currentness è una relazione verificabile tra file, versione, dati e autorità. Quando il legame manca, si mantiene HOLD per le azioni dipendenti e si ricostruisce il

riferimento corretto.

4. Git non annulla ogni effetto

Git consente di tracciare e recuperare codice, ma non ripristina da solo un database, una email già inviata o una transazione finanziaria. Una modifica al codice può essere annullata mentre i dati restano nello stato introdotto dalla migrazione. Per questo l'identità del candidato include, quando necessario, schema, migrazioni, configurazione e artefatti esterni.

Prima di un comando distruttivo si deve sapere esattamente quali file saranno coinvolti e a chi appartengono le modifiche locali. Non usate reset forzati o cancellazioni di interi checkout come risposta ordinaria a un errore. Un revert mirato o un fix possono essere più sicuri se preservano la storia e permettono di verificare l'effetto.

Il commit Git non prova l'identità operativa di un agente. La persona che ha integrato una modifica può essere diversa da chi ha scritto il codice e da chi lo ha autorizzato. Le regole del framework mantengono separate storia del repository, responsabilità ed evidenza di esecuzione.

Anche i worktree temporanei devono essere trattati con attenzione. Sono utili per una reale concorrenza, ma non devono accumularsi senza proprietario. Solo dopo aver verificato che non contengano lavoro corrente o prove canoniche possono essere rimossi e la metadata Git ripulita.

5. Fallimenti ripetuti e cause sistemiche

Un bug ordinario appartiene a I3 nell'ambito della campagna valida. Se un test fallisce, il controller investiga e corregge. Il Planner non deve emettere un nuovo charter ogni volta che l'esecuzione incontra una difficoltà.

Tuttavia, riparazioni ripetute che risolvono soltanto il sintomo sono un segnale importante. Immaginiamo che una ricerca soci fallisca alternativamente con accenti, maiuscole e spazi. Dopo diverse correzioni, I3 dovrebbe esaminare l'intero percorso di normalizzazione, persistenza, query e interfaccia, cercando la causa comune.

La revisione indipendente deve mettere alla prova le ipotesi dell'implementatore. Non basta leggere il nuovo test che l'autore ha appena scritto. Occorre cercare casi che possano falsificare la correttezza della soluzione. Il risultato deve essere integrato nello stesso lavoro autorizzato, salvo vera invalidazione strategica.

Anche un valutatore può essere difettoso. Potrebbe accettare un risultato che non soddisfa il prodotto oppure bloccare indefinitamente una capacità per una formalità non più rilevante. La riparazione del valutatore è un'attività tecnica, ma non deve rimuovere silenziosamente un controllo di sicurezza necessario.

Il criterio resta il rischio reale. Un sistema che corregge la causa riduce il numero di rientri inutili; un sistema che produce altri PASS per gli stessi errori accumula soltanto attività.

6. Sicurezza, privacy e diritti di terzi

Se un incidente riguarda dati personali, accessi o segreti, bisogna limitare l'esposizione preservando le informazioni necessarie a capire il guasto. Non copiate password, token o dati reali in Issue pubbliche o log condivisi senza protezione.

Il gestionale dell'esempio contiene nomi e quote. Per testare il codice si dovrebbero usare dati fittizi o adeguatamente anonimizzati. L'invio di dati reali a un servizio esterno richiede una valutazione separata delle regole applicabili e dell'autorità del CLIENT. La semplice disponibilità tecnica di un endpoint non costituisce consenso.

L'effetto di una simulazione è diverso da un'azione reale. Generare una bozza di email non equivale a inviarla. Mostrare una ricevuta fittizia non prova un pagamento bancario. Preparare un'immagine non significa aver ottenuto il diritto di pubblicarla. Un sistema affidabile conserva queste differenze sia nel codice sia nei rapporti.

Per materiali di terzi rispettate le licenze effettive e le limitazioni del caso. La licenza MIT dello starter non estende automaticamente i propri termini a ogni biblioteca, video, immagine o servizio utilizzato dal prodotto.

7. Dimostrare che il recupero è avvenuto

La scomparsa del messaggio di errore non basta. Dopo una correzione si verifica l'effetto che prima falliva e le proprietà che potrebbero essere state danneggiate. Per una migrazione, questo include schema, dati, vincoli, eventuali duplicati e comportamento al riavvio. Per una schermata, può essere necessaria una prova reale del flusso dell'utente.

La verifica deve riferirsi a un candidato preciso. Se dopo i test viene modificato il codice, i risultati precedenti non sono automaticamente applicabili. Il rapporto di recupero descrive ciò che è stato realmente osservato e ciò che resta escluso.

Un incidente chiuso correttamente lascia uno stato corrente coerente: problema, causa, correzione, prove e prossimo passo. Le vecchie note restano evidenza storica, non una seconda fonte di verità attuale. Se una parte dell'effetto non è verificabile, il limite va dichiarato.

In caso di dubbio su una transazione esterna già eseguita, non la si ripete alla cieca. Prima si cerca una ricevuta o un riferimento idempotente e si coinvolge l'autorità competente. La prudenza è particolarmente necessaria quando l'azione è irreversibile.

8. Tre casi di recupero completi

Caso A — Dipendenza Python assente. Lo scaffold è riuscito, ma la validazione indica che manca jsonschema. L'esecutore controlla l'interprete

attivo e requirements-validation.txt, installa la dipendenza dichiarata e ripete il controllo. Non modifica requisiti, Product State né charter. Il recupero riguarda l'ambiente, non il prodotto.

Caso B — Migrazione interrotta. Durante una prova su dati sintetici, l'operazione si interrompe. I3 ferma nuove scritture, esamina journal e stato delle transazioni, confronta schema e backup. Solo dopo decide se completare, ripetere in sicurezza o ripristinare. Il successo è verificato con test di integrità e riavvio; non viene dichiarato sulla base di una semplice frase «riprovato».

Caso C — Rapporto di accettazione vecchio. Un documento dice che la ricerca soci è stata approvata, ma da allora il codice è cambiato. Il Planner conserva la precedente decisione come storia e segnala che la currentness è venuta meno. I3 identifica il nuovo candidato e ripete i gate interessati. Se necessario, il CLIENT prova nuovamente il flusso visibile.

Questi esempi richiedono strategie diverse. Un unico comando «reinstalla tutto» non risolve simultaneamente ambiente, banca dati e identità delle prove. La diagnosi per causa è più efficace di una sequenza automatica di tentativi.

9. Operazioni sensibili e decisioni corrette

Azione	Gestione ordinaria	Confine importante
Leggere log locali	Diagnosi tecnica in sola lettura	Non divulgare segreti
Ripetere un test unitario	I3 nella campagna	Collegarlo al candidato
Ripristinare una banca dati di prova	I3 dopo controllo di stato	Destinazione esatta e ripetibilità
Eliminare dati produttivi	Recupero esplicitamente autorizzato	Valutare perdita e backup
Ripetere un pagamento reale	Non automaticamente	Verificare effetto precedente e permesso
Pubblicare contenuti di terzi	Verifica di diritti e condizioni	Uso specifico e licenza
Cambiare una richiesta confermata	CLIENT e Planner quando serve	Registrare la supersessione
Sostituire un charter con un nuovo nome	Non è una riparazione valida	Ricostruire l'autorità reale

La tabella non sostituisce la valutazione di una situazione concreta. Aiuta a riconoscere le operazioni che hanno effetti oltre il codice e che non possono essere autorizzate implicitamente da una richiesta generica di completare il lavoro.

10. Limiti di una validazione riuscita

Un test del framework può essere verde mentre l'applicazione non si avvia. Un test della funzione di calcolo può passare mentre l'interfaccia seleziona il socio sbagliato. Un video può sembrare convincente mentre il database perde i dati al riavvio. Sono problemi diversi e richiedono prove diverse.

Mantenete separati i gate: unità, integrazione, DB/currentness, semantica e generalizzazione, funzionamento fisico, qualità percettiva e release. Non tutti i progetti richiedono ogni possibile verifica nella stessa profondità, ma un gate rilevante non può essere sostituito silenziosamente da un altro.

Un HOLD preciso non equivale al fallimento totale del progetto. Indica che una certa affermazione o azione non è ancora giustificata. Si possono proseguire verifiche indipendenti che non dipendono dall'informazione mancante. Questa granularità permette di lavorare senza fingere certezza.

L'accettazione del prodotto resta distinta dai test del pacchetto ArquitecturaRepo. Un buon framework aiuta a raccogliere le prove, ma non può garantire risultati reali per un'applicazione che non ha mai eseguito.

11. Feedback locale e configurazione della privacy

La distribuzione v1.4 può generare localmente un piccolo rapporto di metadati sull'uso del framework. Non include automaticamente un collettore esterno verso il maintainer. La pianificazione con GitHub Actions e l'esecuzione del generatore locale sono controlli separati.

L'opzione –no-feedback-workflow disattiva la pianificazione periodica ma non il generatore. Per evitare qualsiasi rapporto, impostate enabled=false in .agentic-sdlc/feedback/ARCHITECTURE_FEEDBACK_CONFIG.json. Per i dettagli consultate la politica sul feedback.

Se qualcuno segnala una trasmissione indesiderata, verificate il percorso realmente eseguito, la configurazione e il contenuto del rapporto. Non deducete l'invio dal solo fatto che un file esista e non deducete l'assenza di ogni esecuzione dalla mancanza di un scheduler.

Una cultura del recupero matura protegge i dati, preserva le prove e rimuove le cause dei problemi senza moltiplicare inutilmente ruoli o procedure. Il suo successo è una ripresa controllata, non una dashboard piena di indicatori verdi.

Segue: capitolo 06, Accettazione delle capacità.

Fonti e limiti

- Currentness e ricostruzione
- Recupero transazionale
- Sicurezza e azioni esterne
- Igiene del repository

- Integrità delle revisioni
- Privacy del feedback

Gli incidenti illustrati sono fittizi e non autorizzano alcuna operazione su dati reali di terzi.

06 — Accettazione e chiusura delle capacità

> Una dichiarazione di PASS non basta a concludere una capacità. Serve un candidato esatto, con evidenze appropriate e l’approvazione delle autorità corrette. Questo capitolo segue il ciclo di vita delle capacità e l’integrità della revisione terminale.

Argomenti

1. Una capacità è un risultato di prodotto
2. Cinque componenti della chiusura
3. Identificare esattamente il candidato
4. Lacune di prova senza nuove campagne artificiali
5. Accettare, continuare o cambiare obiettivo
6. Revisioni periodiche e regola di arresto
7. Un rapporto di chiusura verificabile
8. Un rifiuto che non obbliga a ricominciare
9. Software utilizzabile e test dello starter
10. Sapere quando concludere la sessione

1. Una capacità è un risultato di prodotto

Un endpoint, una tabella e un modulo software sono parti della soluzione. Una capacità è ciò che l’utente può fare grazie a quelle parti. Nell’applicazione dell’associazione, per esempio, la capacità «registrare una quota e ritrovarla nello storico» attraversa interfaccia, validazione, permessi e persistenza.

Il codice può compilare senza essere utile. Una tabella può esistere senza alcuna schermata per consultarne i dati. Un’operazione può funzionare una volta ma duplicare un pagamento se viene ripetuta. La conclusione della capacità deve quindi riguardare il percorso concordato e i suoi principali rischi, non semplicemente la presenza di file.

L’ambito resta importante. Se la prima versione non prevede pagamenti online, il team non deve bloccare indefinitamente la quota locale perché non è stata integrata una banca. Allo stesso tempo, non può sostenere che la funzione locale sia pronta se perde i dati al riavvio o consente a utenti non autorizzati di alterare le registrazioni.

Il requisito di prodotto chiarisce il risultato desiderato. Il charter stabilisce ciò che è autorizzato. Le prove dimostrano che un candidato specifico lo realizza. L’accettazione corretta collega tutte queste parti senza confonderne i proprietari.

2. Cinque componenti della chiusura

Un buon rapporto terminale include cinque elementi. Il primo è la capacità e la sua origine: requisito confermato e charter valido. Il secondo è l'identità del candidato: codice, configurazione e dati pertinenti. Il terzo è l'insieme delle verifiche tecniche, semantiche e fisiche richieste. Il quarto è la revisione indipendente e, quando serve, la valutazione umana. Il quinto è la decisione corrente con rischi e prossima azione.

Non sono cinque documenti necessariamente distinti. Un progetto piccolo può usare riferimenti brevi a fonti già esistenti. L'obiettivo non è moltiplicare i PDF o le Issue, ma permettere a una persona nuova di capire esattamente che cosa è stato approvato.

Ogni prova ha una portata. Il test di una funzione copre certe condizioni; un'integrazione prova il passaggio tra componenti; un controllo del database valuta la persistenza; una sessione con l'utente osserva la comprensibilità. Un singolo gate non deve assorbire automaticamente gli altri.

Se manca una verifica importante, il rapporto deve dirlo. Un candidato verificato soltanto in locale con dati sintetici può essere un buon prototipo, ma non si dovrebbe presentarlo come sistema di produzione collaudato. L'onestà dell'affermazione è parte della qualità del prodotto.

3. Identificare esattamente il candidato

Il candidato sottoposto a revisione non è «l'ultima versione» in senso generico. Può essere un commit preciso, una configurazione determinata, una versione del database e un insieme di artefatti correlati. Se il branch cambia durante o dopo la prova, bisogna riconciliare quali risultati restino applicabili.

Il review freeze serve a proteggere questa identità. Un revisore indipendente esamina lo stesso contenuto che l'esecutore propone di accettare. Se una modifica importante viene effettuata dopo il review, il candidato cambia e le verifiche interessate vanno ripetute.

Immaginiamo che la funzione di quota sia provata sul commit A. Dopo il test, I3 modifica il modo di gestire la doppia registrazione nel commit B. Il vecchio risultato può restare una fonte storica, ma non certifica automaticamente il nuovo comportamento. Serve almeno una verifica adeguata della logica cambiata e delle sue interazioni.

Neppure il solo Git dimostra che il database corrente corrisponda alle migrazioni del codice. Se la capacità dipende da dati persistenti, la relazione va controllata. Un candidato può essere formalmente identificato e comunque non avere evidenza sufficiente per un gate di currentness.

La responsabilità di chi integra il commit e l'identità dell'agente che ha eseguito il lavoro sono concetti diversi. I progetti che richiedono tracciabilità di esecuzione devono conservarla sulle superfici pertinenti, senza desumerla dalla sola cronologia Git.

4. Lacune di prova senza nuove campagne artificiali

Un difetto ordinario o un test mancante, già compreso nel charter, torna a I3. Il controller corregge, aggiunge le verifiche e presenta un candidato aggiornato. Non serve una nuova campagna strategica ogni volta che un revisore trova un problema.

Se la funzione quote supera i test unitari ma non è stata avviata nell'applicazione reale, manca il gate fisico. I3 esegue il flusso e documenta il risultato nella stessa campagna. Se emerge un errore di persistenza, lo corregge e ripete i test pertinenti. Questo è lavoro operativo normale.

Se lo stesso sottosistema fallisce più volte, la risposta deve diventare sistemica. Non basta correggere una eccezione per iterazione. I3 esamina l'intero percorso, i dati, gli errori e la concorrenza, poi richiede una revisione indipendente più ampia prima di un nuovo rapporto terminale.

Una vera invalidazione del charter è diversa. Se il CLIENT decide improvvisamente che i pagamenti devono passare da un provider bancario reale, cambiano responsabilità, sicurezza, costi e requisiti. Questo richiede una decisione di prodotto e una valutazione strategica. Non è una semplice mancanza di test.

5. Accettare, continuare o cambiare obiettivo

Le conclusioni possibili possono essere raggruppate in tre categorie. **Accettare** se il candidato corrente soddisfa l'intenzione confermata e i gate necessari sono dimostrati. **Continuare** se restano difetti o prove da completare all'interno dei limiti autorizzati. **Cambiare la direzione** se emerge una nuova esigenza o un rischio materiale incompatibile con il charter.

Queste strade producono azioni diverse. Accettare significa aggiornare lo stato canonico e rendere chiaro che cosa è stato effettivamente ottenuto. Continuare significa restituire falsificatori specifici a I3. Cambiare la direzione significa coinvolgere il CLIENT e il Planner per chiarire la nuova capacità e le sue conseguenze.

Un rifiuto tecnico ben formulato descrive un problema riproducibile. «In due richieste simultanee la quota viene salvata due volte; il test di integrazione sul candidato mostra il duplicato» è utile. «Non mi convince, ricominciamo» non identifica una causa; può però giustificare una prova umana se la difficoltà riguarda davvero l'esperienza.

Un'accettazione con limitazioni deve riportarle. «Funziona in locale con dati sintetici, non è pubblicato» è una conclusione onesta. «Pronto per qualunque ambiente» non segue da quelle prove. È meglio un confine chiaro che un'etichetta apparentemente definitiva ma ingannevole.

6. Revisioni periodiche e regola di arresto

La qualità richiede rigore, non perfezionismo senza fine. Se tutti i gate realmente applicabili sono stati soddisfatti e non esistono falsificatori materiali, continuare a cercare microdifetti teorici può consumare tempo senza migliorare il prodotto.

Il Planner può eseguire verifiche periodiche della direzione, delle dipendenze e dei rischi. Questo non significa ripetere una nuova analisi strategica completa ogni volta che I3 incontra un test fallito. I passaggi strategici principali hanno uno scopo distinto dalla normale iterazione tecnica.

Dopo più ritorni dello stesso tipo, la strategia operativa deve cambiare: hardening completo del sottosistema, revisione indipendente e nuovo candidato complessivo. Continuare ad accumulare correzioni marginali senza esaminare la causa non è una dimostrazione di qualità.

Misurate il progresso nelle capacità utilizzabili, nei gate superati e nei rischi effettivamente ridotti. Più commit, più documenti o più ore di esecuzione possono essere strumenti utili, ma non sono indicatori sufficienti di successo.

Il CLIENT deve poter valutare se la funzione risolve il problema reale. Un test può essere perfetto nel proprio ambito ma non rilevare che l'utente non capisca un pulsante o che il report risponda alla domanda sbagliata. La verifica percettiva ha un proprietario e uno scopo diversi.

7. Un rapporto di chiusura verificabile

Per l'esempio delle quote, un riepilogo potrebbe essere:

Campo	Evidenza attesa
Capacità	Registrare una quota locale e ritrovarla
Confine	Nessun pagamento bancario esterno
Candidato	Commit, configurazione e database esatti
Unit	Importo valido, campi obbligatori e arrotondamenti
Integration	Modulo, servizio e persistenza insieme
DB/currentness	Migrazione applicata al database verificato
Semantic	Nessuna doppia registrazione e corretto storico
Physical	Applicazione avviata nell'ambiente previsto
Perceptual	Utente capace di completare il percorso
Release	Pacchetto e autorizzazioni di pubblicazione distinti
Rischi	Solo quelli realmente rimasti, con proprietario

Le righe sono un modello didattico, non l'affermazione che i test siano stati eseguiti. In un rapporto reale ogni gate riporta la prova effettiva e il candidato a cui si riferisce. Non basta scrivere PASS nella casella.

Un controllo esterno può decidere che una parte dei gate non si applica a una capacità particolare, ma la motivazione deve essere coerente con il rischio. Per esempio, una modifica a un testo non richiede la stessa verifica di un nuovo

schema di pagamenti. Non bisogna però nascondere una lacuna significativa dietro una formula generica.

La chiusura include la prossima azione consentita. Dopo la corretta registrazione delle quote, il progetto potrebbe passare a un rapporto mensile. Se invece il database resta incerto, il passo corretto è chiarirne lo stato prima di nuove scritture.

8. Un rifiuto che non obbliga a ricominciare

Un revisore scopre che il doppio click su «Salva quota» crea due record. Il charter richiedeva una singola registrazione per operazione. Il difetto è quindi dentro il perimetro autorizzato.

I3 analizza eventi UI, richieste, gestione dei retry, transazioni e chiavi di idempotenza. Dopo la correzione crea casi di test che riproducono il problema e dimostrano che la seconda richiesta non provoca un nuovo effetto indebito. Il revisore indipendente esamina il candidato aggiornato.

Il Planner riceve il rapporto consolidato quando una decisione strategica è matura. Non deve formulare un nuovo charter per ogni esperimento sulla stessa causa. Il CLIENT non deve scegliere se usare una transazione serializzabile o un indice unico, a meno che una conseguenza di prodotto non richieda la sua opinione.

Se durante l’analisi emerge una richiesta di riscuotere denaro reale tramite un provider, si tratta di una nuova questione. La sua autorizzazione e il relativo rischio non possono essere dedotti dal charter delle quote registrate localmente.

9. Software utilizzabile e test dello starter

La CI di ArquitecturaRepo può verificare correttamente il pacchetto e i suoi contratti. Questi controlli non certificano tutte le future applicazioni sviluppate con esso. Un controller che recupera uno stato sintetico non prova che un gestionale reale sopravviva alla perdita di un disco.

Anche un’applicazione presentata in un video può sembrare pronta pur non avendo prove di persistenza o autorizzazione. Ogni evidenza deve essere valutata per ciò che mostra. La coerenza delle affermazioni è più importante della quantità di screenshot o rapporti.

Una capacità è davvero utile quando il percorso dell’utente funziona nel contesto previsto e i rischi proporzionati sono stati affrontati. Il framework serve a rendere riconoscibile questo risultato, non a sostituire la verifica.

10. Sapere quando concludere la sessione

Una campagna può essere chiusa dopo un’accettazione corretta e la registrazione del suo risultato corrente. Una sessione quotidiana può

terminare anche con la campagna ancora aperta, purché siano disponibili checkpoint, evidenze e azione successiva.

Il CLIENT non deve essere continuamente presente per gestire test e correzioni. Se manca una vera decisione di prodotto, si ferma soltanto la parte che ne dipende. Il silenzio dell'utente non significa autorizzazione a spendere o pubblicare, ma nemmeno un ordine implicito di sospendere ogni altra attività già permessa.

Una nuova sessione dovrebbe poter ricostruire la capacità accettata, il candidato, le verifiche, i problemi rimasti e l'autorità del prossimo passo. Se ciò riesce, il sistema ha preservato un risultato oltre la singola conversazione.

Segue: capitolo 07, Esempio completo di associazione.

Fonti e limiti

- Ciclo di vita ed evidenze
- Integrità della revisione
- Charter di campagna
- Execution Team OS
- Gate della release
- Ricostruzione del contesto

La tabella è un modello e non sostituisce le prove reali del progetto che si intende consegnare.

07 — Esempio completo: gestione di un’associazione

ArquitecturaRepo · Manuale completo per l’utente · Edizione v1.4 · Italiano

> Questo capitolo segue un prodotto fittizio dalla prima richiesta all’accettazione. Tutti i soci, le quote e i risultati citati sono esempi didattici. Il progetto di riferimento dello starter verifica alcuni meccanismi portabili, non costituisce prova di un’applicazione reale già consegnata.

Percorso

1. Il prodotto e i suoi utenti
2. Le domande che cambiano la soluzione
3. Un’installazione senza confusione tra repository
4. Prima capacità: anagrafica soci
5. Dal Planner alla campagna eseguibile
6. Verifiche prima dell’accettazione
7. Seconda capacità: registrare le quote
8. Valutare l’intero percorso
9. Primo ciclo con dati fittizi
10. Gestire un difetto e la sua responsabilità
11. Il secondo ciclo dipende dal primo
12. Esercizio finale di ricostruzione

1. Il prodotto e i suoi utenti

Immaginiamo una piccola associazione sportiva che gestisce qualche centinaio di soci. L’amministrazione utilizza fogli di calcolo per nomi, contatti, iscrizioni e quote. Più persone possiedono copie diverse dei file e talvolta si registrano pagamenti duplicati. Il consiglio vuole conoscere gli arretrati; l’operatore desidera una ricerca semplice e affidabile.

Il CLIENT definisce la prima versione: «Voglio un programma locale per gestire soci e quote. Lo userà una persona dell’amministrazione. Per ora niente pagamento online e niente applicazione mobile. La cronologia deve essere conservata anche quando un socio non è più attivo».

Questo brief contiene già alcune decisioni importanti. Il sistema deve essere locale e semplice; la conservazione dei pagamenti è funzionale, non un dettaglio tecnico; servizi bancari esterni sono esclusi. Altre questioni restano aperte: autorizzazioni precise, durata delle informazioni e intervallo dei backup.

La squadra non dovrebbe inventare queste risposte. Il Planner verifica il

contesto reale, chiarisce i punti materiali e propone una prima capacità consegnabile. Il cliente non è responsabile della struttura delle tabelle, ma deve poter valutare il risultato che userà.

La qualità ricercata è concreta. Una segreteria deve trovare una persona, modificarne i dati e controllare le quote senza leggere documentazione tecnica. Un'interfaccia complicata può vanificare anche una struttura dati corretta.

2. Le domande che cambiano la soluzione

Durante la discovery il Planner fa domande pertinenti:

- Chi può creare o modificare un socio?
- Quali dati sono indispensabili?
- Due persone possono avere lo stesso nome?
- Qual è il significato di disattivazione?
- Le quote precedenti restano consultabili dopo la disattivazione?
- I pagamenti sono registrazioni locali o transazioni vere?
- Quanto deve essere veloce la ricerca?
- Che cosa succede se il computer si guasta?
- Quali funzioni devono essere disponibili nella prima dimostrazione?

Supponiamo che il CLIENT confermi una sola postazione amministrativa, identità stabile per ogni socio, storico dei pagamenti e nessun servizio esterno. Conferma anche che i duplicati devono essere segnalati. L'intervallo di backup resta da definire: viene registrato come decisione aperta, senza trasformarlo in un valore arbitrario.

Il Planner interpreta questi requisiti nel contesto tecnico. Una identità stabile evita di collegare quote a persone sbagliate dopo una modifica del nome. La storicità suggerisce che disattivare non debba cancellare automaticamente le registrazioni. Il lavoro offline influenza la scelta della persistenza e del deployment.

Queste sono conseguenze tecniche delle decisioni confermate. Le parti che dipendono da preferenze o autorizzazioni ulteriori devono essere riportate al CLIENT. L'obiettivo non è fargli scegliere una query SQL, ma evitare che la soluzione decida silenziosamente una regola di prodotto.

3. Un'installazione senza confusione tra repository

La squadra dispone del repository pubblico dello starter ArquitecturaRepo e di un repository separato gestione-soci. Il primo contiene protocolli, strumenti e documentazione; il secondo è il prodotto da realizzare. È essenziale non chiedere al coding agent di aggiungere i moduli del gestionale direttamente al repository pubblico del framework.

Lo starter è stato identificato tramite la release v1.4.0 e il checksum del pacchetto. Il ramo main può contenere libri e guide pubblicati più tardi, che non facevano parte dello ZIP iniziale.

Dato che l'applicazione conserva dati persistenti, il profilo STATEFUL è una scelta ragionevole. Nel terminale PowerShell del framework, per un progetto nuovo non ancora governato:

```
& "python" "tools\scaffold_project.py" --target "..\gestione-soci" --
profile "STATEFUL" --product-id "GESTIONE_SOCI" --objective "Anagrafica e quote d
-language "it" --no-feedback-workflow
```

Il comando è un esempio, non un'operazione già eseguita. Il percorso deve essere controllato prima della scrittura. Se gestione-soci possiede già una struttura .agentic-sdlc/ con stato valido, si segue la verifica in sola lettura invece di sovrascriverla.

Dopo l'installazione, nella cartella dell'applicazione:

```
& "python" -m pip install -r ".agentic-sdlc\requirements-validation.txt"
&   "python"   ".agentic-sdlc\tools\project_status.py"   --control-
root ".agentic-sdlc"
&   "python"   ".agentic-sdlc\tools\discover_project.py"   --project-
root "."
& "python" ".agentic-sdlc\tools\reconstruct_context.py" --root ".agentic-
sdlc"
```

HOLD / UNTRUSTED_CONTEXT può apparire correttamente. Il framework ha creato la struttura ma non ha ancora adottato le informazioni reali del prodotto. Il Planner raccoglie codice, dati, decisioni e baseline tecnica, poi promuove uno stato coerente tramite il processo di adozione.

4. Prima capacità: anagrafica soci

Il primo charter riguarda una capacità circoscritta e utile: creare un socio, ritrovarlo, aggiornare le informazioni e disattivarlo senza perdere la sua identità storica. Una prima capacità così definita permette di testare un flusso reale prima di aggiungere quote e report.

I requisiti osservabili potrebbero essere:

Operazione	Risultato atteso	Caso limite
Creazione	Socio memorizzato con ID stabile	Campi obbligatori mancanti
Ricerca	Persona ritrovata per nome o codice	Omonimi e caratteri speciali
Modifica	Dati aggiornati anche dopo riavvio	Conflitti di aggiornamento pertinenti
Disattivazione	Stato inattivo senza cancellare collegamenti storici	Dati già associati a pagamenti

Operazione	Risultato atteso	Caso limite
Errore	Messaggio chiaro e operazione non distruttiva	Salvataggio interrotto o duplicato

Si tratta di criteri proposti, non di test già eseguiti. Il CLIENT conferma il comportamento e il Planner collega le condizioni alla baseline. Engineering sceglie componenti, schema, validazione e strategia di persistenza.

Una implementazione potrebbe separare accesso ai dati, logica dei soci e interfaccia, ma il framework non obbliga a una struttura di classi unica. L'importante è che le responsabilità siano coerenti e che il comportamento possa essere verificato e ripreso.

I3 deve testare sia il caso normale sia le situazioni che cambiano significato: due soci con lo stesso nome non sono necessariamente duplicati; un codice già utilizzato potrebbe non essere valido; una disattivazione non dovrebbe eliminare quote storiche. Questi casi derivano direttamente dall'intenzione di prodotto.

5. Dal Planner alla campagna eseguibile

Il Planner considera il percorso end-to-end, le dipendenze, i rischi sui dati e le autorizzazioni. Svolge la propria analisi strategica e una revisione avversariale distinta quando l'importanza del charter lo richiede. La revisione cerca ipotesi fragili: sicurezza insufficiente, ambiguità sulla cancellazione, mancanza di recupero o gate non adeguati.

Il charter descrive risultato, confini, input, interfacce, responsabilità e criteri. Non deve trasformarsi in una lunga serie di istruzioni su ogni funzione interna. Una consegna eseguibile potrebbe essere:

> Obiettivo: rendere utilizzabile l'anagrafica soci locale. Stato iniziale: repository e requisiti correnti verificati. Confini: solo dati sintetici nei test, nessun servizio esterno, nessun deployment pubblico, nessuna modifica non autorizzata a pagamenti reali. Verifiche: unità, integrazione, database, semantica e prova effettiva dell'interfaccia come controlli distinti. Risultato terminale: candidato identificato e rapporto con prove e limiti.

Una volta autorizzato, I3 possiede la realizzazione tecnica. Può scegliere l'ordine delle attività, correggere bug, ripetere i test e richiedere una revisione indipendente. Non deve ritornare al Planner per ogni dettaglio di implementazione. La separazione evita che il CLIENT diventi il coordinatore dei tentativi tecnici.

Il controller opzionale dello starter può aiutare a mantenere alcuni stati, ma non certifica automaticamente l'esecuzione di agenti remoti o l'accettazione del prodotto. Le prove devono provenire dalle operazioni realmente compiute nell'applicazione.

6. Verifiche prima dell’accettazione

La prima verifica riguarda i componenti singoli: controllo dei dati obbligatori, creazione di identificatori, cambio di stato e gestione degli errori. Seguono test di integrazione che percorrono interfaccia, logica e database, includendo la rilettura di informazioni dopo un riavvio.

Il gate DB/currentness verifica che schema e migrazioni corrispondano al candidato attuale. Il gate semantico cerca errori di significato: un socio disattivato non deve diventare una persona inesistente se la cronologia richiede ancora il riferimento.

La prova fisica avvia l’applicazione nel contesto previsto. Un modulo importato in un test Python non è la stessa cosa di un programma che l’amministrativo riesce a utilizzare. La verifica percettiva guarda leggibilità, passaggi, messaggi di errore e comprensione dell’operazione.

Supponiamo che i dati si salvino correttamente ma l’utente non riceva alcuna conferma visibile. Il risultato potrebbe essere tecnicamente valido e comunque produrre doppi click o errori d’uso. Se il charter prevedeva una interazione comprensibile, I3 corregge il difetto nella stessa campagna.

Le prove devono riferirsi al candidato esatto. Se il codice cambia dopo il review, si ripetono i controlli influenzati dalla modifica. Non basta riutilizzare una pagina con la parola PASS.

7. Seconda capacità: registrare le quote

Una volta verificata l’identità stabile dei soci, il Planner può proporre la gestione delle quote. La nuova capacità dipende dalla precedente: ogni pagamento locale deve appartenere a una persona corretta e rimanere collegato anche in caso di modifica del nome o dello stato.

Il CLIENT definisce importi, scadenze, eventuali pagamenti parziali, correzioni consentite e responsabilità di chi registra le quote. Il team tecnico stabilisce lo schema, la logica delle transazioni e le verifiche necessarie. Le banche e i servizi di pagamento veri restano esclusi dalla prima versione.

Un rischio importante è la duplicazione. Se l’operatore clicca due volte o si verifica un retry, non deve comparire automaticamente un doppio pagamento. La capacità richiede quindi un comportamento idempotente o comunque una protezione equivalente, progettata e provata da Engineering.

La disattivazione di un socio non cancella i suoi pagamenti passati. Questa regola collega il secondo charter al primo e mostra perché le capacità non sono semplici cartelle isolate. Quando una funzione precedente cambia, i suoi consumatori devono essere rivalutati in modo proporzionato.

Una volta implementata la quota, l’amministrativo deve poterla registrare, rivedere e correggere secondo regole esplicite. Un report degli arretrati può venire dopo, quando si conosce il significato esatto di pagato, scaduto e parziale.

8. Valutare l’intero percorso

La dimostrazione più convincente unisce le capacità: creare un socio, cercarlo, registrare una quota, ritrovare lo storico, disattivare la persona e verificare che i pagamenti restino consultabili. Questo percorso rivela problemi di integrazione che test isolati potrebbero non vedere.

Il Planner controlla il rapporto tra requisiti e risultato. La base tecnica verifica integrità dei dati, autorizzazioni e ripresa. Il CLIENT prova ciò che realmente farà ogni giorno. Una schermata gradevole non sostituisce la sicurezza; una transazione corretta non sostituisce la comprensione umana.

Il progetto deve inoltre conoscere i propri limiti. Un sistema locale provato con qualche centinaio di soci non è automaticamente pronto per migliaia di accessi concorrenti via Internet. Una quota registrata in un database non è una transazione bancaria riuscita.

L’accettazione deve nominare il candidato, l’ambiente e i gate superati. Le limitazioni restano visibili. In questo modo la prossima modifica potrà partire da un fatto verificabile invece che da una promessa generica.

9. Primo ciclo con dati fittizi

Create tre persone sintetiche: Socio A attivo, Socio B con un cognome uguale a quello di A e Socio C destinato alla disattivazione. Inserite dati corretti e dati incompleti. Cercate per nome e codice, modificate un contatto e verificate il risultato dopo aver chiuso e riaperto il programma.

Provate anche caratteri speciali e omonimi. Se un test fallisce, preservate il caso riproducibile e lasciate che I3 corregga la causa nel charter. Non è necessario riscrivere la strategia di prodotto perché la normalizzazione di una stringa era sbagliata.

Usate un database di prova e, se previsto dalla baseline, esercitate una procedura di recupero. Controllate che i dati siano davvero persistenti. Un test che si limita a restituire un oggetto in memoria non prova il comportamento del prodotto dopo il riavvio.

Prima dell’accettazione finale, una persona prova i passaggi. Se non riesce a interpretare il pulsante di disattivazione o non capisce un errore, il difetto va registrato con riferimento al candidato. La qualità dell’interazione è parte del risultato.

10. Gestire un difetto e la sua responsabilità

Immaginiamo tre problemi. Primo: la ricerca fallisce con le lettere accentate. Si tratta normalmente di un difetto tecnico nel comportamento già concordato; I3 corregge. Secondo: l’associazione vuole improvvisamente iscrizioni pubbliche via Internet. È un cambiamento di prodotto che richiede nuova valutazione strategica. Terzo: dopo un’interruzione non si sa se una quota sia

stata già salvata. È una questione di stato e idempotenza: ulteriori scritture dipendenti restano sospese finché non viene chiarita.

Questa classificazione determina il lavoro corretto. Un bug non comporta automaticamente un nuovo Goal. Un requisito completamente nuovo non va nascosto in un refactoring. Un'informazione tecnica sconosciuta non deve essere inventata per rendere verde il rapporto.

Se I3 ha già corretto più volte lo stesso tipo di errore, deve rivedere l'intero sottosistema e richiedere una verifica indipendente. Il problema potrebbe essere nella strategia di confronto, nella persistenza o nei test stessi. Fare una nuova microcampagna ogni giorno non risolve la causa.

Il CLIENT interviene quando l'esito visibile o l'autorità riservata richiedono una decisione. Non deve coordinare le prove di ogni fix né fornire una soluzione tecnica pronta.

11. Il secondo ciclo dipende dal primo

La funzione quote si appoggia all'identità dei soci già accettata. Il controller verifica che la base corrente corrisponda a quella su cui si vuole costruire. Se la precedente capacità è stata modificata, può essere necessaria una prova di integrazione prima di procedere.

Si registra una quota sintetica per Socio A. Il programma viene riavviato e la quota resta visibile. La stessa richiesta viene ripetuta per verificare che il risultato non venga duplicato. Socio C viene disattivato e lo storico resta consultabile secondo il requisito.

Un revisore controlla i casi difficili: importo invalido, registrazione ripetuta, errore di salvataggio, autorizzazione e collegamento al socio corretto. Il CLIENT prova il percorso end-to-end. Solo con evidenze adeguate e correntezza del candidato la capacità può essere accettata.

Il report successivo del consiglio dipenderà dalla definizione di arretrato. Una query che somma correttamente un insieme di dati può comunque dare un risultato non utile se la regola economica è ambigua. È il motivo per cui la corretta collaborazione fra CLIENT, Planner e I3 conta più della sola velocità di generazione del codice.

12. Esercizio finale di ricostruzione

Per verificare che il framework abbia realmente migliorato la continuità, immaginate di consegnare il repository a una nuova sessione senza il vecchio dialogo. Questa sessione deve poter rispondere:

1. Qual è l'obiettivo del gestionale e chi lo usa?
2. Quali requisiti sono stati confermati e quali restano aperti?
3. Quale versione del codice e del database è corrente?
4. Quali capacità sono state realmente accettate?
5. Quali test e prove umane sostengono l'accettazione?
6. Quali autorizzazioni esterne mancano?

7. Quale campagna è aperta e chi può proseguire?
8. Qual è la prossima azione consentita e quale non lo è?

Se la risposta richiede una ricostruzione manuale di tutta la chat, cercate la fonte mancante: requisiti, currentness, rapporto di review, proprietario del dato o prossimo passo. Correggete la superficie canonica, non soltanto il riepilogo.

Lo scopo non è convincere l'utente con una quantità enorme di file. È rendergli possibile concentrarsi sulle scelte di prodotto, mentre Engineering può sviluppare, verificare, integrare e riprendere il lavoro in modo affidabile.

Fonti e limiti

- Riferimento stateful e trasferimento
- Installazione pubblica v1.4
- Attachment di progetti governati
- Discovery e baseline
- Adozione
- Standard del charter
- Gates ed evidenze
- Currentness

I dati e gli esiti dell'associazione sono fittizi. Questa dimostrazione non autorizza operazioni reali né afferma che esista un gestionale già prodotto.